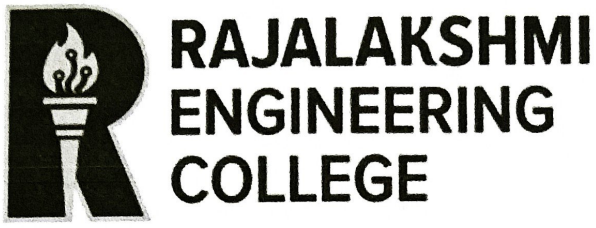
RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 – Fundamentals of Data Science Lab
Laboratory Record Note Book

Name : ....NETHRA. B.....

Year / Branch / Section : .....II - CSE.....

University Register No. : ...2116250701479.....

College Roll No. : .....250701479.....

Semester : .....III.....

Academic Year : .....2026 - 2027.....

DEPARTMENT OF COMPUTER SCIENCE

AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	NETHRA . B
ROLL NO	2116250701479
YEAR	II
SEC	

RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution

BONAFIDE CERTIFICATE

Name: .....NETHRA....B.....

Academic Year: 2026-2027. **Semester:** ...III..... **Branch:** ..B.E...CSE

Register No. 2116 250701479

Certified that this is the bonafide record of work done by the above student in the...FUNDAMENTALS.....OF....DATA...SCIENCE..... *Laboratory during the academic year 2026 – 2027.*

3/11/26

Signature of Faculty in-charge

Submitted for the Practical Examination held on ..06/11/2026.....

Internal Examiner

External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	**https://github.com/nethra-b22/FDS---Lab-experiments**
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Unstructured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	

3.a	Conduct an experiment to show data visualization using line plot	19/08/26
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26
5	Data Collection and Initial Exploration	16/09/26
6	Time Series Analysis of Website Traffic	16/09/26

7	Random Sampling and Sampling Distribution	23/09/26	
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	**https://github.com/nethra-b22/FDS---Lab-experiments**
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDAQuantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	**https://github.com/nethra-b22/FDS---Lab-experiments**

19	Experiment to understand Logistic Regression for a given data set.	07/10/26	**https://github.com/nethra-b22/FDS---Lab-experiments**
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	**https://github.com/nethra-b22/FDS---Lab-experiments**

Experiment -1

1.a. Analyze the trend of data

science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib for visualization.

PROGRAM:-

```
import pandas as pd

import matplotlib.pyplot as plt

data ={'Year': list(range(2010,2021)),

    'Job posting':[150,300,450,600,800,1200,1600,2100,2700,3400,4200]}

df=pd.DataFrame(data)

plt.plot(df['Year'],df['Job posting'],marker='o')

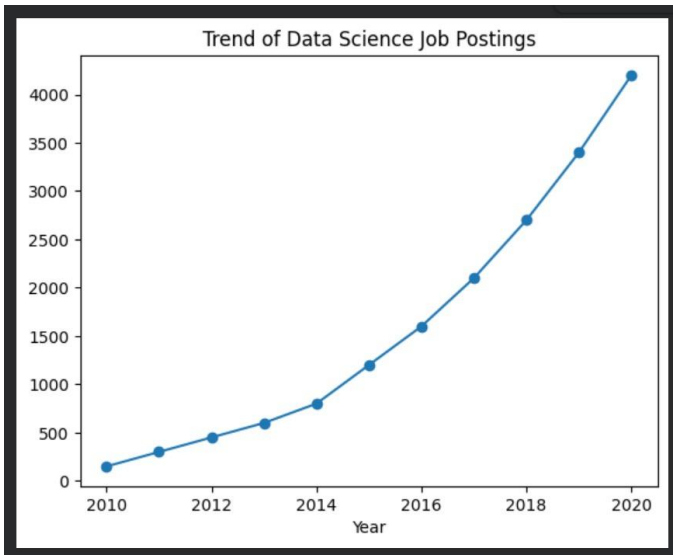
plt.title("Trend of Data Science Job Postings")

plt.xlabel('Year')

plt.ylabel('Number of Job Postings')

plt.show()
```

OUTPUT:-



1.b.Analyze
and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

PROGRAM:-

```
import pandas as pd

import matplotlib.pyplot as plt

roles=['Data Analyst','Data Engineer','Data Scientist','ML Engineer','Business Analyst']

counts=[300,500,450,200,150]

plt.bar(roles,counts)

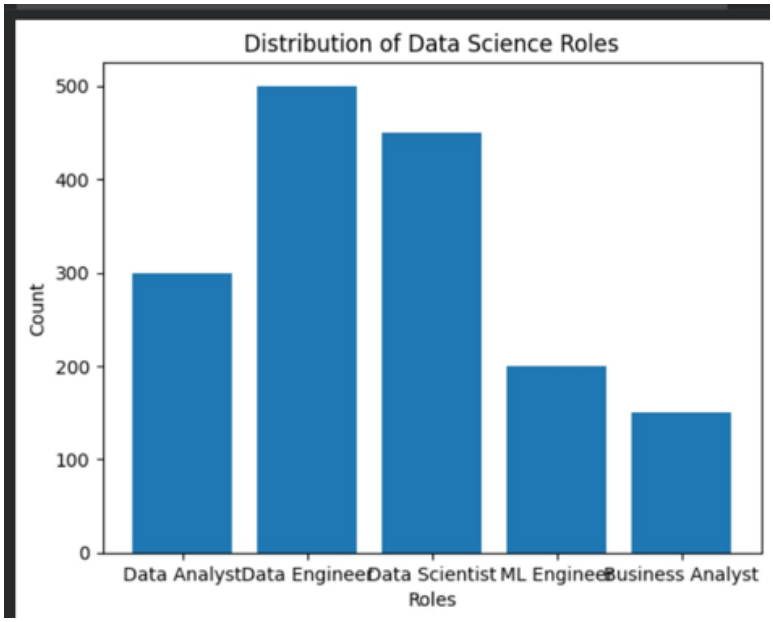
plt.title('Distribution of Data Science Roles')

plt.xlabel('Roles')

plt.ylabel('Count')

plt.show()
```

OUTPUT:-



1.c.Conduct

an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

PROGRAM:-

```
import pandas as pd

structured_data=pd.DataFrame({

  'ID':[1,2,3],

  'Name':['Alice','Bob','Charlie'],

  'Age':[25,30,35]

})

print("Structured Data:\n",structured_data)

unstructured_data="This is an example of unstructured data.it can be a piece of text, an image, or a video file."

print("\nUnstructured Data:\n",unstructured_data)

semi_structured_data={'ID':1,'Name':'Alice','Attributes':{'Height':165,'Weight':68}}

print("\nSemi-structured Data:\n",semi_structured_data)
```

OUTPUT:-

Structured Data:

	ID	Name	Age
0	1	Alice	25
1	2	Bob	30
2	3	Charlie	35

Unstructured Data:

This is an example of unstructured data.it can be a piece of text, an image, or a video file.

Semi-structured Data:

{'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}

1.d.Conduct

an experiment to encrypt and decrypt given sensitive data using the cryptography library fernet.

PROGRAM:-

```
Import pandas as pd

from cryptography.fernet import Fernet

key = Fernet.generate_key()

f = Fernet(key)

token = f.encrypt(b"Rajalakshmi Engineering College")

token

b'...'

f.decrypt(token)

b'Rajalakshmi Engineering College'

key = Fernet.generate_key()

cipher_suite = Fernet(key)

plain_text = b"Rajalakshmi Engineering College."

cipher_text = cipher_suite.encrypt(plain_text)

decrypted_text = cipher_suite.decrypt(cipher_text)

print("Original Data:", plain_text)

print("Encrypted Data:", cipher_text)

print("Decrypted Data:", decrypted_text)
```

OUTPUT:-

```
Original Data: b'Rajalakshmi Engineering College.'

Encrypted Data: b'gAAAAABqfKxkWnnrfj9EG1MdQ8V5lo-
bltGUGZtSmxnPlf_5NqAxYxh3nRoOQcF_h-
nSVSpst6B6XnhQh2SehFrCdKAZrcm8MGcKc8sfA4R536lgLnjS47smFgZnHnAWJ8r9ZNKVK4lb'

Decrypted Data: b'Rajalakshmi Engineering College.'
```

EXPERIMENT – 2

Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

PROGRAM:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded=files.upload()
df=pd.read_csv('SALES DATA - SALES DATA.csv')
print(df.head())
print(df.isnull().sum())
df['Sales']=(df['Sales'].fillna(df['Sales'].mean))
df.dropna(subset=['Product','Quantity','Region'],inplace=True)
print(df.describe())
product_summary=df.groupby('Product').agg({
    'Sales':'sum',
    'Quantity':'sum'
}).reset_index()
print(product_summary)
plt.figure(figsize=(10,6))
plt.bar(product_summary['Product'],product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date']=pd.to_datetime(df['Date'],format='%d-%m-%Y')
sales_over_time=df.groupby('Date').agg({'Sales':'sum'}).reset_index()
plt.figure(figsize=(10,6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()
pivot_table=df.pivot_table(values='Sales',index='Region',columns='Produ
ct',aggfunc='sum',fill_value=0)
print(pivot_table)
correlation_matrix=df.select_dtypes(include='number').corr()
print(correlation_matrix)
import seaborn as sns
plt.figure(figsize=(10,6))
sns.heatmap(correlation_matrix,annot=True,cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

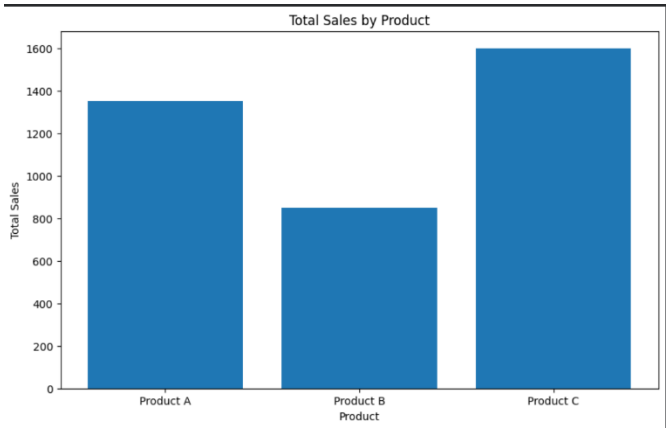
OURPUT:-

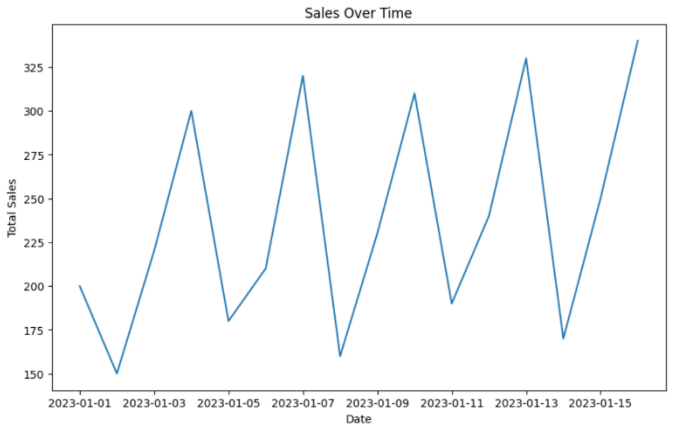
	Date	Product	Sales	Quantity	Region
0	01-01-2023	Product A	200	4	North
1	02-01-2023	Product B	150	3	South
2	03-01-2023	Product A	220	5	North
3	04-01-2023	Product C	300	6	East
4	05-01-2023	Product B	180	4	West

```
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
```

	Sales	Quantity
count	16.000000	16.000000
mean	237.500000	5.375000
std	64.031242	1.746425
min	150.000000	3.000000
25%	187.500000	4.000000
50%	225.000000	5.500000
75%	302.500000	7.000000
max	340.000000	8.000000

	Product	Sales	Quantity
0	Product A	1350	33
1	Product B	850	17
2	Product C	1600	36

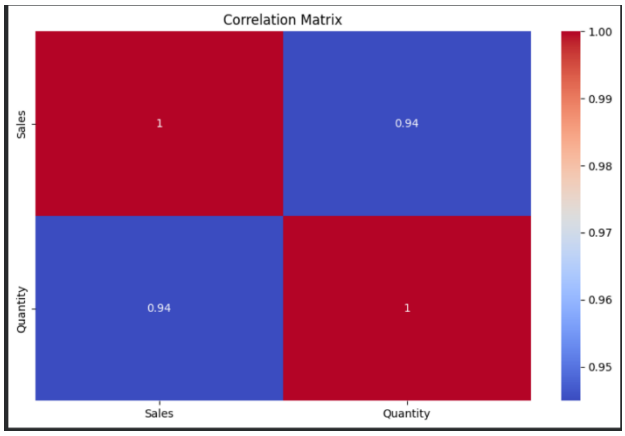




Product	Product A	Product B	Product C

Region			
East	0	0	1600
North	1350	0	0
South	0	480	0
West	0	370	0

	Sales	Quantity
Sales	1.000000	0.944922
Quantity	0.944922	1.000000



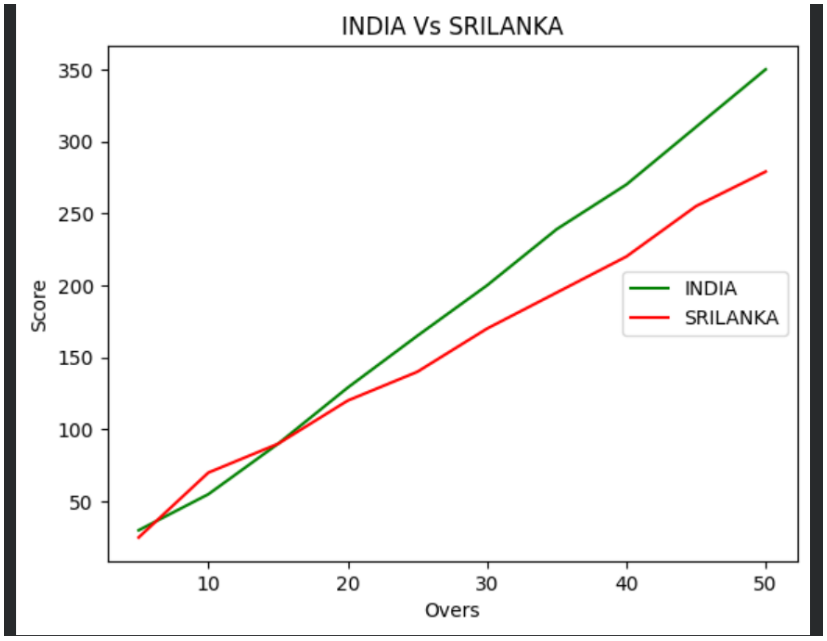
EXPERIMENT NO: 03

3 a) Conduct an experiment to show data visualization using line plot.

PROGRAM:-

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

OUTPUT:-

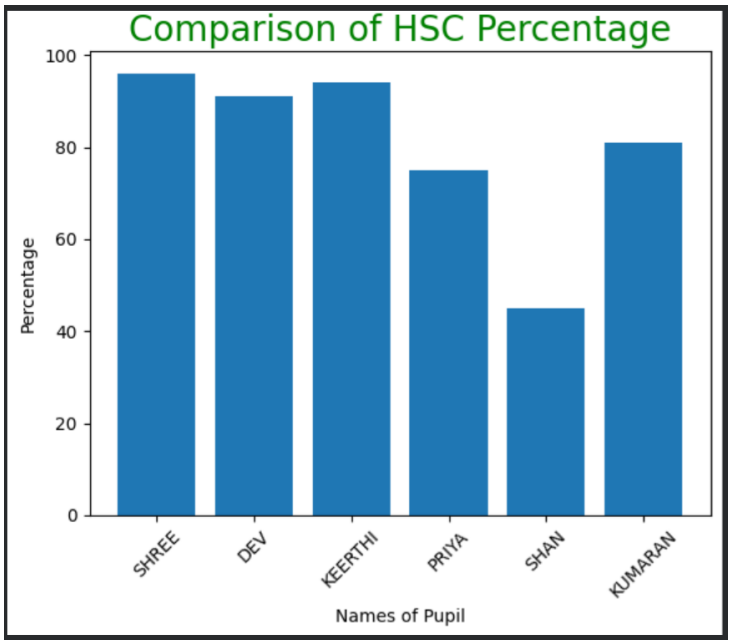

INDIA Vs SRILANKA
Score
Overs
INDIA
SRILANKA
350
300
250
200
150
100
50
10
20
30
40
50

3 b) Conduct an experiment to show data visualization using bar chart.

PROGRAM:-

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

OUTPUT:-

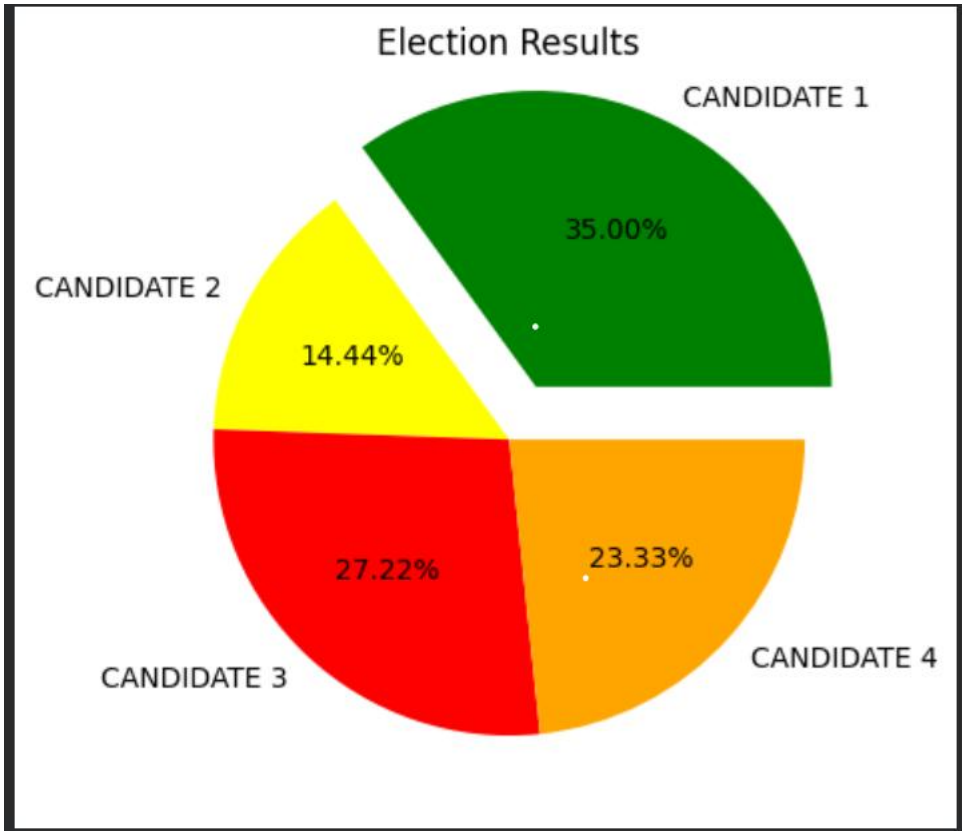


3 c) Conduct an experiment to show data visualization using pie chart

PROGRAM:-

```
import matplotlib.pyplot as election
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```

OUTPUT:-



EXPERIMENT NO : 04

Count the frequency of occurrence of a word in a body of text is often needed during text processing.

PROGRAM:-

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)
wlist = []
for i in range(50):
   wlist.append(token[i])
wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT:-

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma
', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever'
, 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1
), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposi
tion', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1
), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('e
xistence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly
', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world'
, 1), ('with', 2)]
```

EXPERIMENT NO: 05

Data Collection and Initial Exploration

PROGRAM:-

1. Data Collection:

```
import pandas as pd
db=pd.read_csv("diabetess.csv")
print(db)
```

OUTPUT:-

```
     Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0              6      148             72             35        0  33.6
1              1       85             66             29        0  26.6
2              8      183             64              0        0  23.3
3              1       89             66             23       94  28.1
4              0      137             40             35      168  43.1
..           ...      ...            ...            ...      ...   ...
763           10      101             76             48      180  32.9
764            2      122             70             27        0  36.8
765            5      121             72             23      112  26.2
766            1      126             60              0        0  30.1
767            1       93             70             31        0  30.4

     DiabetesPedigreeFunction  Age  Outcome
0                       0.627   50        1
1                       0.351   31        0
2                       0.672   32        1
3                       0.167   21        0
4                       2.288   33        1
..                        ...  ...      ...
763                     0.171   63        0
764                     0.340   27        0
765                     0.245   30        0
766                     0.349   47        1
767                     0.315   23        0

[768 rows x 9 columns]
```

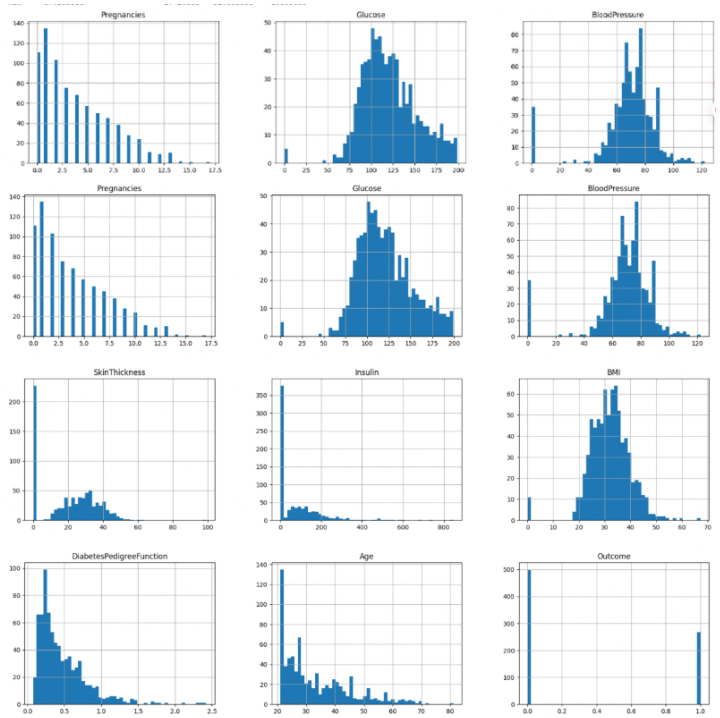
2.Initial Exploration

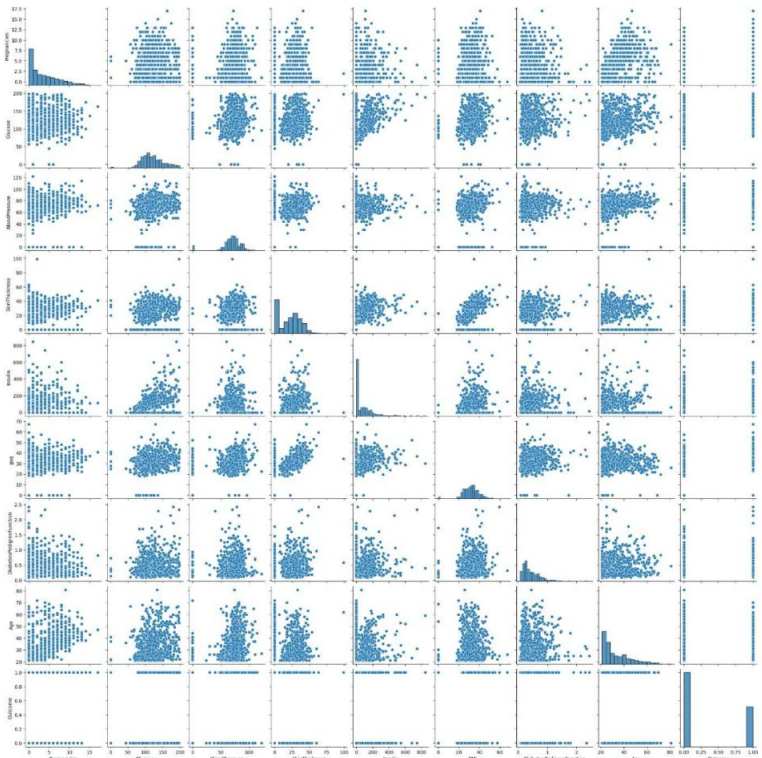
```
print(db.info())
print(db.describe())
import matplotlib.pyplot as plt
import seaborn as sns
db.hist(bins=50,figsize=(20,15))
plt.show()
sns.pairplot(db)
plt.show()
```

OUTPUT:-

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```


Pregnancies
Glucose
BloodPressure
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome



EXPERIMENT NO : 06

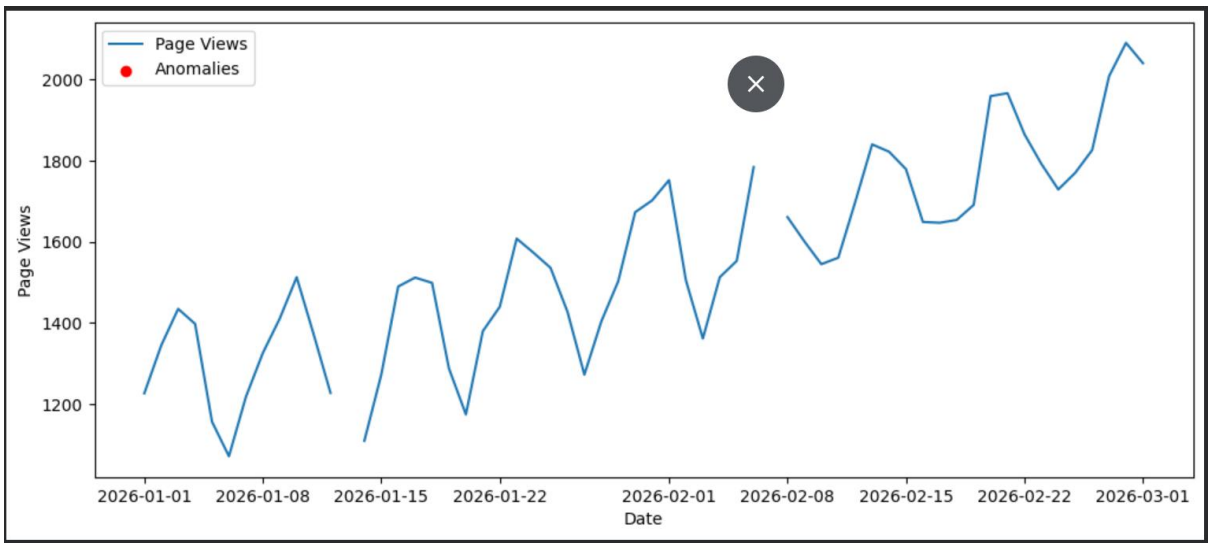
Time Series Analysis of Website Traffic

PROGRAM:-

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv('website_traffic.csv', parse_dates=['Date'])
data.set_index('Date', inplace=True)
data['Page_Views_Z'] = (
   data['Page_Views'] - data['Page_Views'].mean()
) / data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.figure(figsize=(12, 5))

plt.plot(
   data.index,
   data['Page_Views'],
   label='Page Views'
)
plt.scatter(
   anomalies.index,
   anomalies['Page_Views'],
   color='red',
   label='Anomalies'
)
plt.legend()
plt.xlabel('Date')
plt.ylabel('Page Views')
plt.show()
```

OUTPUT:-


Page Views
Anomalies
2000
1800
1600
1400
1200
Page Views
2026-01-01
2026-01-08
2026-01-15
2026-01-22
2026-02-01
2026-02-08
2026-02-15
2026-02-22
2026-03-01
Date

EXPERIMENT NO : 07

Random Sampling and Sampling Distribution

PROGRAM:-

```
import numpy as np
import matplotlib.pyplot as plt
population_mean = 50
population_std = 10
population_size = 100000

population = np.random.normal(
    population_mean,
    population_std,
    population_size
)
sample_sizes = [30, 50, 100]
num_samples = 1000

sample_means = {}

for size in sample_sizes:
    sample_means[size] = []

    for _ in range(num_samples):
        sample = np.random.choice(
            population,
            size=size,
```

```
            replace=False
        )
        sample_means[size].append(np.mean(sample))
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)

    plt.hist(
        sample_means[size],
        bins=30,
        alpha=0.7,
        label=f"Sample Size {size}"
    )

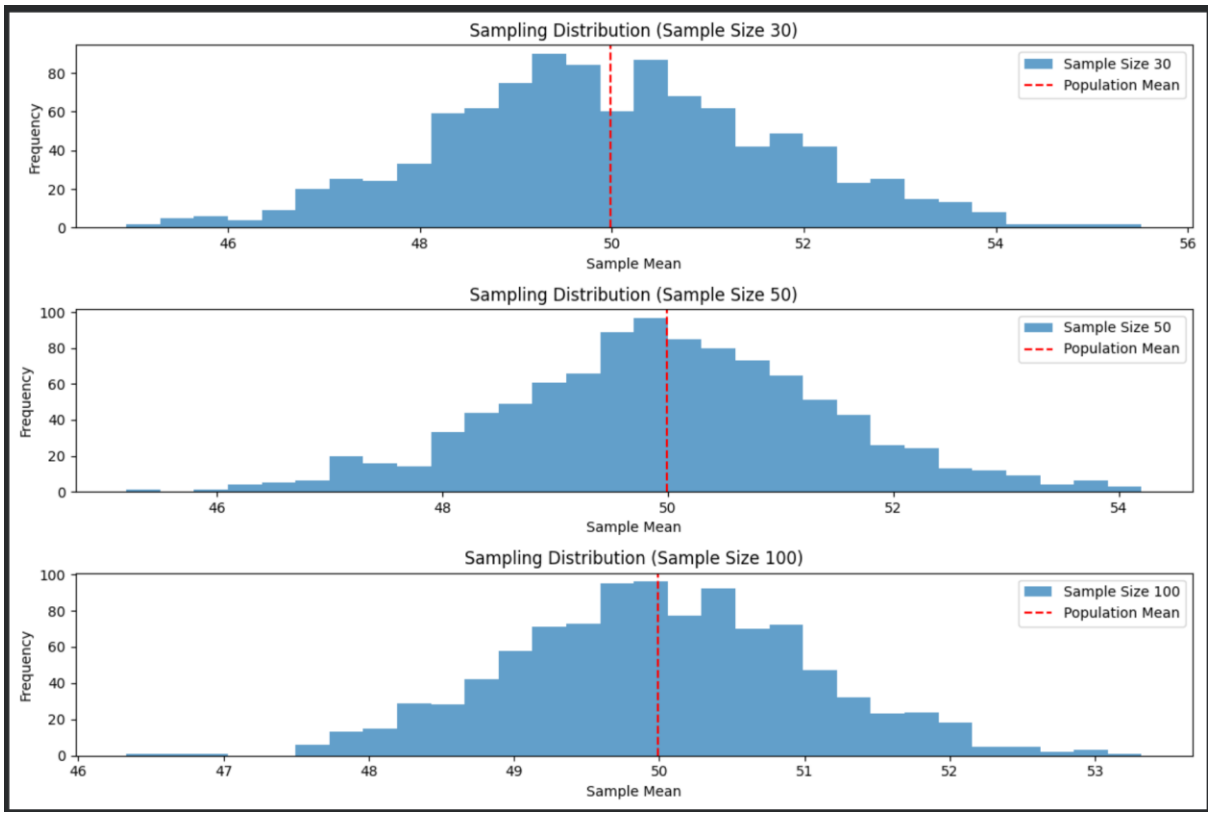
    plt.axvline(
        np.mean(population),
        color="red",
        linestyle="dashed",
        linewidth=1.5,
        label="Population Mean"
    )

    plt.title(f"Sampling Distribution (Sample Size {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
```

```
    plt.legend()


plt.tight_layout()
plt.show()
```

OUTPUT:-



EXPERIMENT NO : 08

Conduct Z test for the Data Given using Python Objective

PROGRAM:-

```
import numpy as np

import scipy.stats as stats

sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149, 151, 150, 149, 152, 151,148, 150, 152, 149, 150, 148, 153, 151, 150, 149, 152, 148, 151, 150, 153])

population_mean = 150

sample_mean = np.mean(sample_data)

sample_std = np.std(sample_data, ddof=1)

n = len(sample_data)

z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))

p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))

print(f"Sample Mean: {sample_mean:.2f}")

print(f"Z-Statistic: {z_statistic:.4f}")

print(f"P-Value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:

   print("Reject the null hypothesis: The average weight is significantly different from 150 grams.")

else:

   print("Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.")
```

OUTPUT:-

```
Sample Mean: 150.20

Z-Statistic: 0.6406

P-Value: 0.5218

Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.
```

EXPERIMENT NO : 09

Experiment to understand Matplotlib library use cases in Data Science through visualization

PROGRAM:-

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
data = pd.read_csv('/content/Iris_Dataset.csv')
data
```

OUTPUT:-

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	variety
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

```
data.info()
```

OUTPUT:-

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

data.describe()

OUTPUT:-

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.057333	3.758000	1.199333
std	43.445368	0.828066	0.435866	1.765298	0.762238
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

data.value_counts('variety')

OUTPUT:-

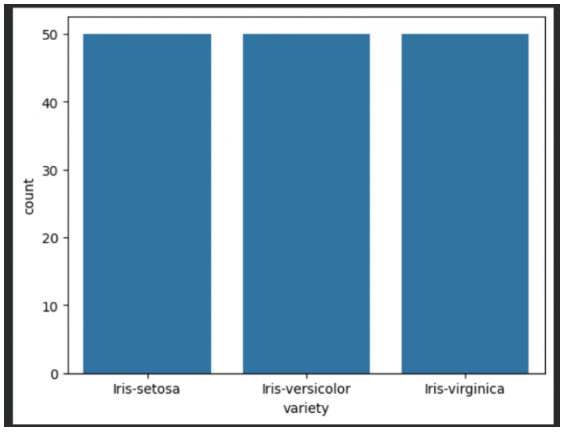
variety	count
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

dtype: int64

sns.countplot(x='variety', data=data)

plt.show()

OUTPUT:-



```
dummies = pd.get_dummies(data.variety)
dummies
```

OUTPUT:-

	Iris-setosa	Iris-versicolor	Iris-virginica
0	True	False	False
1	True	False	False
2	True	False	False
3	True	False	False
4	True	False	False
...	...	...	...
145	False	False	True
146	False	False	True
147	False	False	True
148	False	False	True
149	False	False	True

150 rows × 3 columns

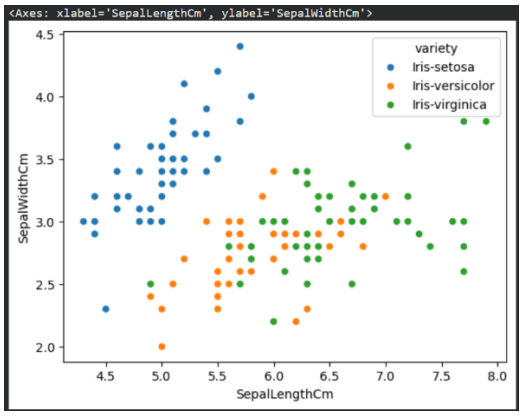
```
FinalDataset = pd.concat(
   [pd.get_dummies(data.variety), data.iloc[:, [0,1,2,3]]],
   axis=1)
FinalDataset.head()
```

OUTPUT:-

	Iris-setosa	Iris-versicolor	Iris-virginica	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm
0	True	False	False	1	5.1	3.5	1.4
1	True	False	False	2	4.9	3.0	1.4
2	True	False	False	3	4.7	3.2	1.3
3	True	False	False	4	4.6	3.1	1.5
4	True	False	False	5	5.0	3.6	1.4

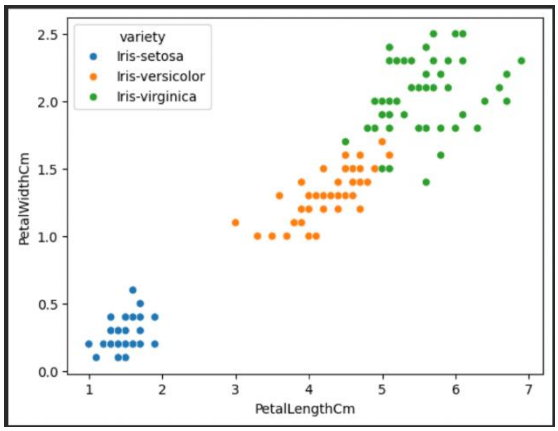
```
sns.scatterplot(x='SepalLengthCm',y='SepalWidthCm',hue='variety',data=data,)
```

OUTPUT:-



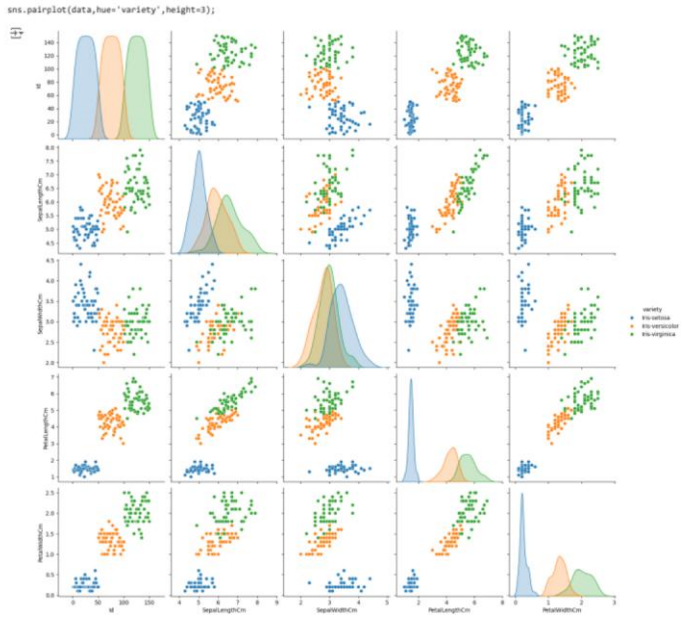

```
sns.scatterplot(
   x='PetalLengthCm',
   y='PetalWidthCm',
   hue='variety',
   data=data)
plt.show()
```

OUTPUT:-



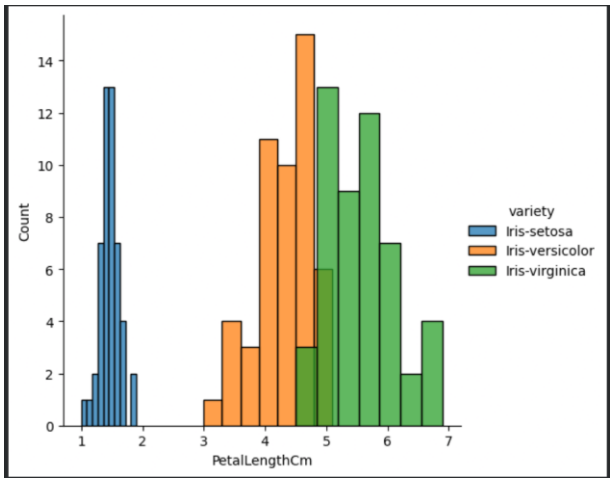
```
sns.pairplot(data, hue='variety', height=3)
plt.show()
```

OUTPUT:-



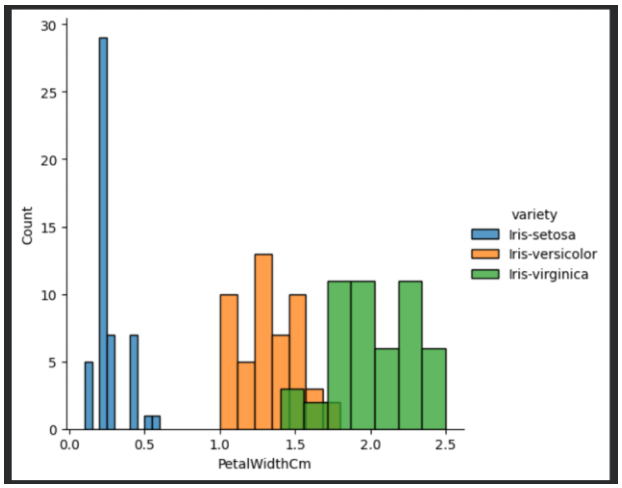

```
sns.FacetGrid(data, hue='variety', height=5).map(
sns.histplot, 'PetalLengthCm').add_legend()
plt.show()
```

OUTPUT:-



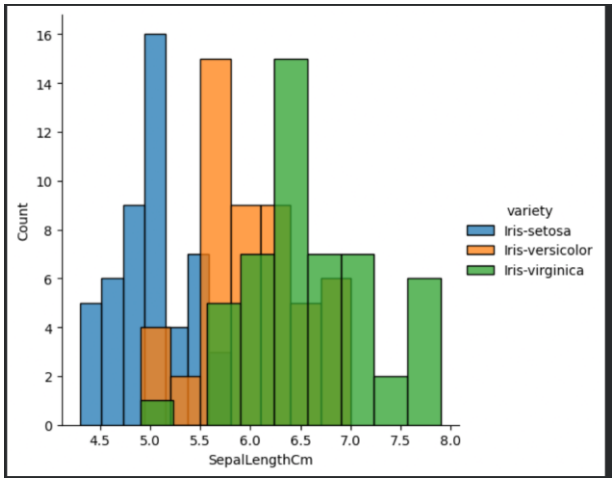
```
sns.FacetGrid(data, hue='variety', height=5).map(
sns.histplot, 'PetalWidthCm').add_legend()
plt.show()
```

OUTPUT:-



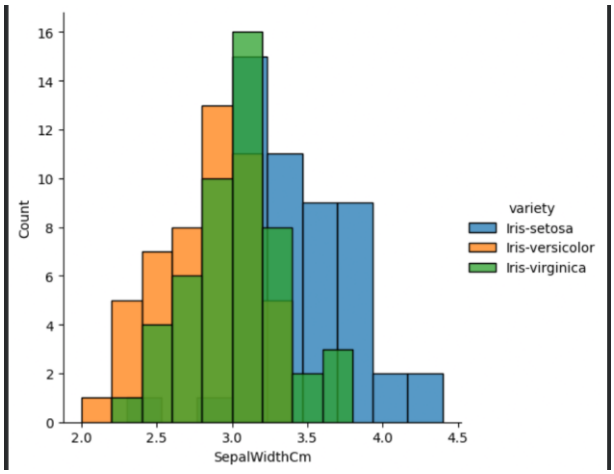

```
sns.FacetGrid(data, hue='variety', height=5).map(
sns.histplot, 'SepalLengthCm').add_legend()
plt.show()
```

OUTPUT:-



```
sns.FacetGrid(data, hue='variety', height=5).map(
sns.histplot, 'SepalWidthCm').add_legend()
plt.show()
```

OUTPUT:-



EXPERIMENT NO : 10

Experiment to understand array function in Data science.

PROGRAM:-

```
import numpy as np
array=np.random.randint(1,100,9)
array
```

OUTPUT:-

```
array([44,  2, 19,  8, 60, 12, 42, 51, 86])
```

```
np.sqrt(array)
```

OUTPUT:-

```
array([6.63324958, 1.41421356, 4.35889894, 2.82842712, 7.74596669,
 3.46410162, 6.4807407 , 7.14142843, 9.2736185 ])
```

```
array.ndim
```

OUTPUT:-

```
1
```

```
new_array=array.reshape(3,3)
new_array
```

OUTPUT:-

```
array([[44,  2, 19],
       [ 8, 60, 12],
       [42, 51, 86]])
```

```
new_array.ndim
2
```

```
new_array.ravel()
```

OUTPUT:-

```
array([44,  2, 19,  8, 60, 12, 42, 51, 86])

newm=new_array.reshape(3,3)
newm
```

OUTPUT:-

```
array([[44,  2, 19],
       [ 8, 60, 12],
       [42, 51, 86]])

newm[2,1:3]
```

OUTPUT:-

```
array([51, 86])

newm[1:2,1:3]
```

OUTPUT:-

```
array([[60, 12]])

new_array[0:3,0:0]
```

OUTPUT:-

```
array([], shape=(3, 0), dtype=int64)

new_array[0:2,0:1]
```

OUTPUT:-

```
array([[44],
       [ 8]])
```

new_array[0:3,0:1]

OUTPUT:-

array([[44],

[8],

[42]])

new_array[1:3]

OUTPUT:-

array([[8, 60, 12],

[42, 51, 86]])

EXP NO:11 EXPERIMENT TO UNDERSTAND PANDAS LIBRARY USE CASES IN DATA SCIENCE.

CODE :

```
import numpy as np
import pandas as pd
list_data = [[1, 'Smith', 50000], [2, 'Jones', 60000]]
df = pd.DataFrame(list_data, columns=['Empd', 'Name', 'Salary'])
print(df)
print(df.info())
df = pd.read_csv("startups_600.csv")
print(df.info())
print(df.head())
print(df.tail())
df = pd.read_csv("employee_1000.csv")
print(df.head())
print(df.tail())
print(df.info())
print(df.salary)
print(type(df.salary))
print(df.salary.mean())
print(df.salary.median())
print(df.salary.mode())
print(df.salary.var())
```

```
print(df.salary.std())
print(df.describe())
print(df.describe(include='all'))
empCol = df.columns
print(empCol)
emparray = df.values
print(emparray)
employee_DF = pd.DataFrame(emparray, columns=empCol)
print(employee_DF)
```

OUTPUT :

```
  Empd   Name  Salary
0    1  Smith   50000
1    2  Jones   60000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
```

```
memory usage: 180.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   R&D Spend       600 non-null    float64
 1   Administration  600 non-null    float64
 2   Marketing Spend  600 non-null    float64
 3   State           600 non-null    object
 4   Profit          600 non-null    float64
dtypes: float64(4), object(1)
memory usage: 23.6+ KB
None
   R&D Spend  Administration  Marketing Spend       State    Profit
0   33787.01       163739.48       478250.70  California  63753.42
1   55718.07        53688.04       368754.18     Florida  77770.43
2   35402.10        53318.53       176625.70     Florida  32413.62
3   17740.51        98541.53       148267.79  California  21578.94
4   24127.17       123296.48       174851.61    New York  49197.95
     R&D Spend  Administration  Marketing Spend       State     Profit
595  174307.36        62718.80       172938.73  California  171958.72
```

```
596  194697.79       157448.51        77229.87  California  149990.84
597  193775.57        60812.65        12723.88    New York  138755.39
598  149930.37        60688.51       322911.19    New York  147532.63
599   26017.25        51816.27       318451.16    New York   58240.98
  emp id                name  salary
0      1  SREYASKARI MULLAPUDI   48000
1      2     SREE VARSSINI K S   48000
2      3           ARUN KUMAR   90000
3      4              PRIYA S   90000
4      5              PRIYA S   75000
    emp id              name  salary
995    996           SWETHA R   90000
996    997          VIGNESH M   28000
997    998           ADITYA R   25000
998    999          ARUN KUMAR   30000
999   1000  SREE VARSSINI K S   28000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  1000 non-null   int64
 1   name    1000 non-null   object
```

```
 2   salary  1000 non-null   int64
dtypes: int64(2), object(1)
memory usage: 23.6+ KB
None
0      48000
1      48000
2      90000
3      90000
4      75000
       ...
995    90000
996    28000
997    25000
998    30000
999    28000
Name: salary, Length: 1000, dtype: int64
<class 'pandas.core.series.Series'>
54982.0
50000.0
0    55000
Name: salary, dtype: int64
474538214.2142142
21783.898049114494
```

```
          emp id         salary
count  1000.000000    1000.000000
mean    500.500000   54982.000000
std     288.819436   21783.898049
min       1.000000   25000.000000
25%     250.750000   35000.000000
50%     500.500000   50000.000000
75%     750.250000   70000.000000
max    1000.000000  100000.000000
          emp id     name         salary
count   1000.000000     1000    1000.000000
unique         NaN       20           NaN
top            NaN  PRIYA S           NaN
freq           NaN       67           NaN
mean     500.500000      NaN   54982.000000
std      288.819436      NaN   21783.898049
min        1.000000      NaN   25000.000000
25%      250.750000      NaN   35000.000000
50%      500.500000      NaN   50000.000000
75%      750.250000      NaN   70000.000000
max     1000.000000      NaN  100000.000000
Index(['emp id', 'name', 'salary'], dtype='object')
[[1 'SREYASKARI MULLAPUDI' 48000]
```

```
 [2 'SREE VARSSINI K S' 48000]
 [3 'ARUN KUMAR' 90000]
 ...
 [998 'ADITYA R' 25000]
 [999 'ARUN KUMAR' 30000]
 [1000 'SREE VARSSINI K S' 28000]]
    emp id                name salary
0        1  SREYASKARI MULLAPUDI  48000
1        2     SREE VARSSINI K S  48000
2        3           ARUN KUMAR  90000
3        4              PRIYA S  90000
4        5              PRIYA S  75000
..     ...                  ...    ...
995    996            SWETHA R  90000
996    997           VIGNESH M  28000
997    998            ADITYA R  25000
998    999           ARUN KUMAR  30000
999   1000     SREE VARSSINI K S  28000

[1000 rows x 3 columns]
```

EXP NO:12 EXPERIMENT TO DETECT OUTLIERS IN A GIVEN DATA SET

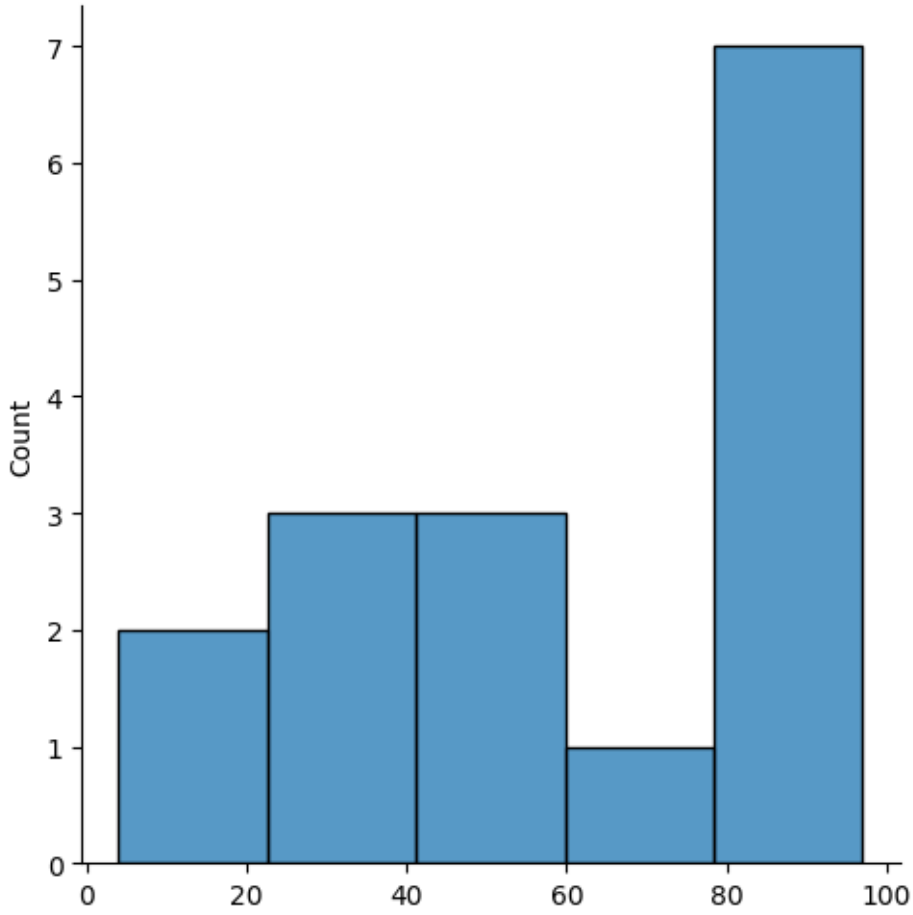
CODE :

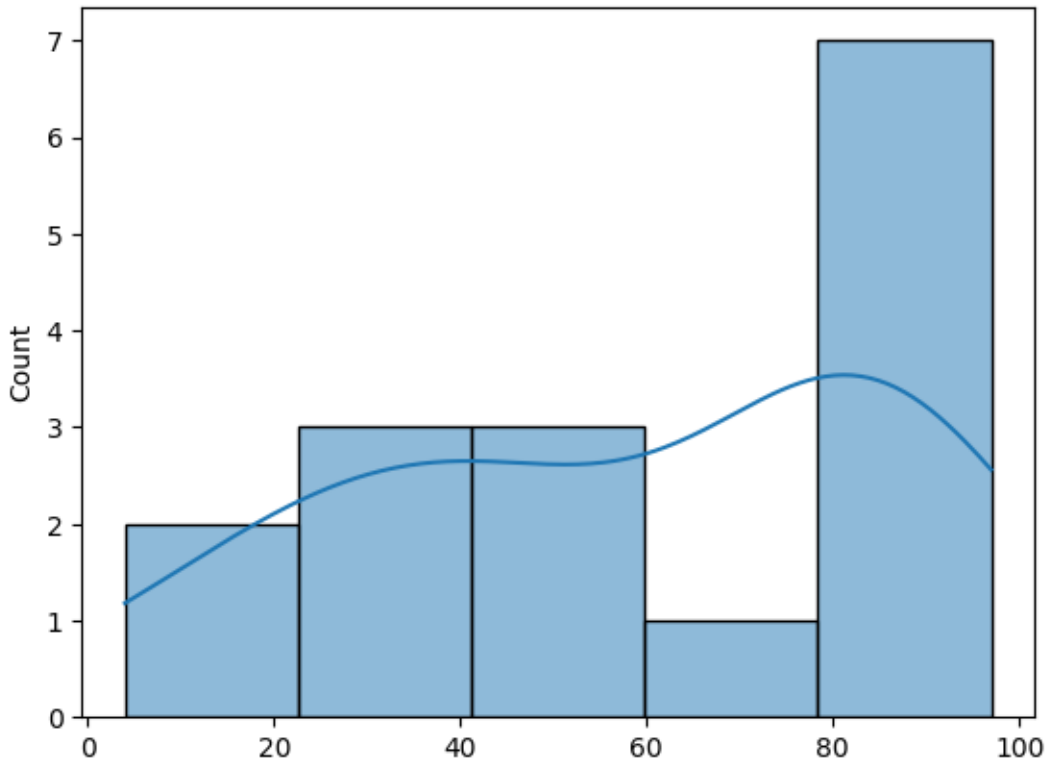
```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
array = np.random.randint(1, 100, 16)
print(array)
print(array.mean())
print(np.percentile(array, 25))
print(np.percentile(array, 50))
print(np.percentile(array, 75))
print(np.percentile(array, 100))
def outDetection(array):
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur
lr, ur = outDetection(array)
print(lr, ur)
sns.displot(array)
plt.show()
```

```
sns.histplot(array, kde=True)  # updated instead of distplot
plt.show()
new_array = array[(array > lr) & (array < ur)]
print(new_array)
sns.displot(new_array)
plt.show()
lr1, ur1 = outDetection(new_array)
print(lr1, ur1)
final_array = new_array[(new_array > lr1) & (new_array < ur1)]
print(final_array)
sns.histplot(final_array, kde=True)
plt.show()
```

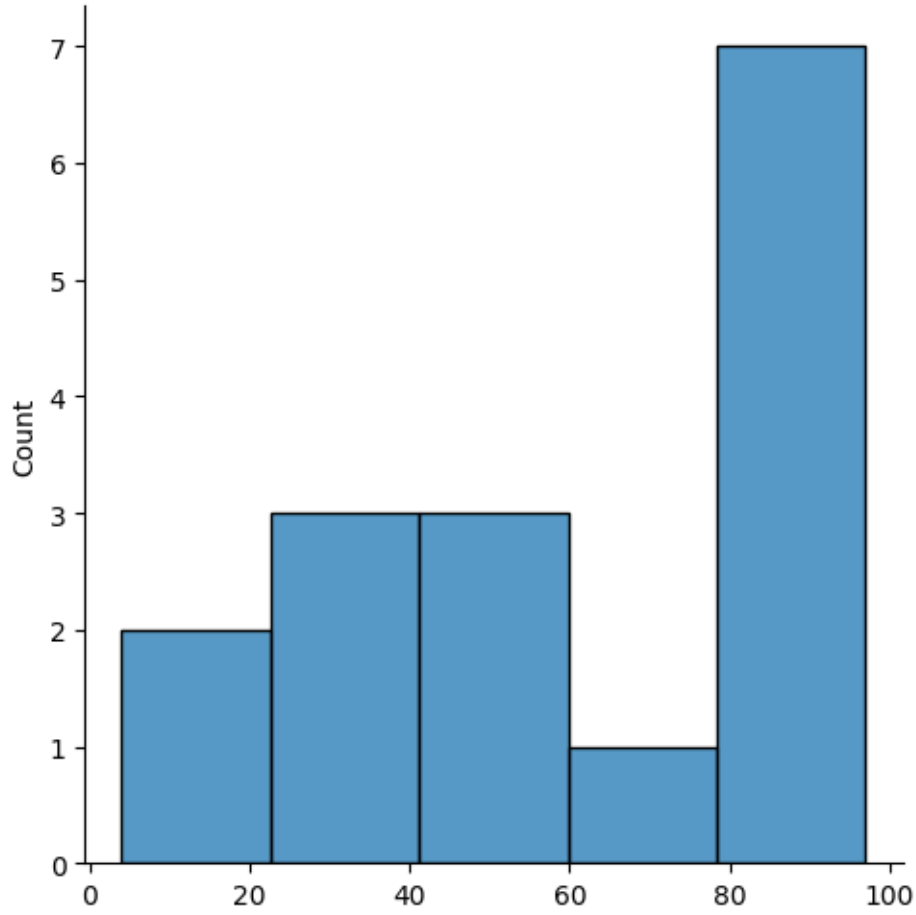
OUTPUT :

```
[23 78 52 21 39 46  4 97 47 80 79 84 82 84 29 89]
58.375
36.5
65.0
82.5
97.0
-32.5 151.5
```



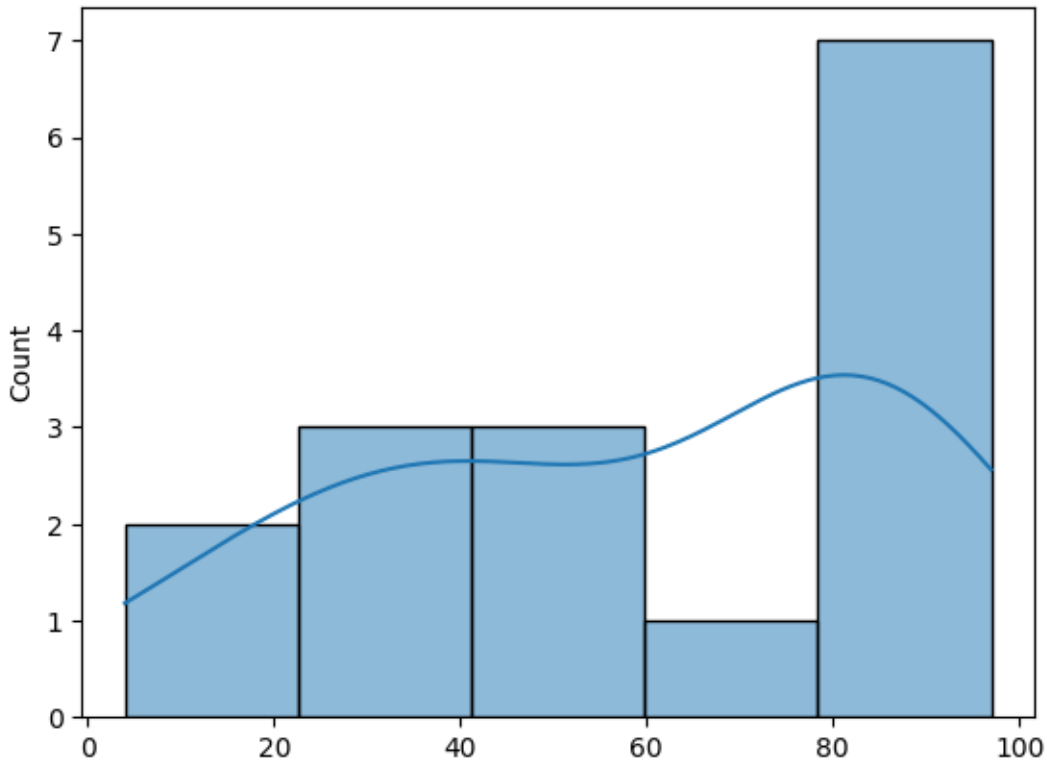


[23 78 52 21 39 46 4 97 47 80 79 84 82 84 29 89]



-32.5 151.5

[23 78 52 21 39 46 4 97 47 80 79 84 82 84 29 89]


Count
0
1
2
3
4
5
6
7
0
20
40
60
80
100

EXP NO:13 EXPERIMENT TO HANDLE MISSING VALUES AND INAPPROPRIATE DATA IN A GIVEN DATA SET

CODE :

```
import numpy as np
import pandas as pd
df = pd.read_csv("Hotel_Dataset.csv")
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)
df.loc[df.CustomerID < 0, 'CustomerID'] = np.nan
df.loc[df.Bill < 0, 'Bill'] = np.nan
df.loc[df.EstimatedSalary < 0, 'EstimatedSalary'] = np.nan
df.loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20), 'NoOfPax'] = np.nan
df["Hotel"] = df["Hotel"].replace("Ibys", "Ibis")
df["FoodPreference"] = df["FoodPreference"].replace(
   {"Vegetarian": "Veg", "veg": "Veg", "non-Veg": "Non-Veg"}
)
df["EstimatedSalary"] =
df["EstimatedSalary"].fillna(round(df["EstimatedSalary"].mean()))
df["NoOfPax"] = df["NoOfPax"].fillna(round(df["NoOfPax"].median()))
df["Rating(1-5)"] = df["Rating(1-5)"].fillna(round(df["Rating(1-
5)"].median()))
df["Bill"] = df["Bill"].fillna(round(df["Bill"].mean()))
print(df)
```

OUTPUT :

C:\Users\Ayyadurai\AppData\Local\Temp\ipykernel_5300\2580639570.py:1: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

df.CustomerID.loc[df.CustomerID<0]=np.nan

df.Bill.loc[df.Bill<0]=np.nan

C:\Users\Ayyadurai\AppData\Local\Temp\ipykernel_5300\2580639570.py:2: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan

C:\Users\Ayyadurai\AppData\Local\Temp\ipykernel_5300\2129877948.py:1: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nanarray(['20-25', '30-35', '25-30', '35+'], dtype=object)

array(['Ibis', 'LemonTree', 'RedFox', 'Ibys'], dtype=object)

<bound method Series.unique of 0 veg

1 Non-Veg

2 Veg

3 Veg

4 Vegetarian

5 Non-Veg

6 Vegetarian

7 Veg

8 Non-Veg

9 non-Veg

Name: FoodPreference, dtype: object>

	CustomerID	Age_Group	Hotel	FoodPreference	NoOfPax	Bill	\
0	1.0	30-35	Ibis	Non-Veg	5.0	9227.0	
1	2.0	35+	LemonTree	Non-Veg	3.0	6203.0	
2	3.0	20-25	RedFox	Veg	6.0	2798.0	

3	4.0	30-35	RedFox	Non-Veg	2.0	6291.0
4	5.0	30-35	RedFox	Veg	3.0	6495.0
..	...	...	...	...	...	...
595	596.0	35+	RedFox	Veg	7.0	7530.0
596	597.0	20-25	Ibis	Veg	7.0	1258.0
597	598.0	30-35	Ibis	Veg	8.0	1282.0
598	599.0	35+	LemonTree	Veg	1.0	2898.0
599	600.0	25-30	LemonTree	Non-Veg	8.0	5135.0

	EstimatedSalary	Rating(1-5)
0	26629.0	1
1	85031.0	1
2	53380.0	1
3	37071.0	5
4	40074.0	5
..	...	...
595	90263.0	2
596	73117.0	1
597	78394.0	1
598	43782.0	2
599	48454.0	5

[600 rows x 8 columns]

EXP NO:14 EXPERIMENT TO UNDERSTAND FEATURE SCALING.

CODE :

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, MinMaxScaler
df = pd.read_csv('pre-process_datasample.csv')
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
age = SimpleImputer(strategy="mean", missing_values=np.nan)
salary = SimpleImputer(strategy="mean", missing_values=np.nan)
features[:, [1]] = age.fit_transform(features[:, [1]])
features[:, [2]] = salary.fit_transform(features[:, [2]])
oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
sc = StandardScaler()
feat_standard_scaler = sc.fit_transform(final_set)
mms = MinMaxScaler(feature_range=(0, 1))
feat_minmax_scaler = mms.fit_transform(final_set)
print("Final Set:\n", final_set)
```

```
print("\nStandard Scaled Features:\n", feat_standard_scaler)
print("\nMinMax Scaled Features:\n", feat_minmax_scaler)
```

OUTPUT :

```
Final Set:
 [[0.0 0.0 0.0 ... 1.0 56.0 62884.06]
 [0.0 0.0 0.0 ... 0.0 53.0 89816.41]
 [0.0 1.0 0.0 ... 0.0 44.0 65339.32052173912]
 ...
 [0.0 0.0 0.0 ... 0.0 55.0 18000.0]
 [0.0 0.0 0.0 ... 0.0 40.0 65339.32052173912]
 [0.0 0.0 0.0 ... 0.0 58.0 65339.32052173912]]

Standard Scaled Features:
 [[-2.86794495e-01 -3.10630372e-01 -4.46139852e-01 ...
2.13437475e+00
   1.04820386e+00 -1.18109252e-01]
 [-2.86794495e-01 -3.10630372e-01 -4.46139852e-01 ... -
4.68521286e-01
   8.25741604e-01  1.17745987e+00]
 [-2.86794495e-01  3.21926022e+00 -4.46139852e-01 ... -
4.68521286e-01
   1.58354850e-01  2.10004088e-15]
```

...

[-2.86794495e-01 -3.10630372e-01 -4.46139852e-01 ... -4.68521286e-01

9.74049772e-01 -2.27723766e+00]

[-2.86794495e-01 -3.10630372e-01 -4.46139852e-01 ... -4.68521286e-01

-1.38261485e-01 2.10004088e-15]

[-2.86794495e-01 -3.10630372e-01 -4.46139852e-01 ... -4.68521286e-01

1.19651202e+00 2.10004088e-15]]

MinMax Scaled Features:

[[0. 0. 0. ... 1. 0.80851064 0.38383841]

[0. 0. 0. ... 0. 0.74468085 0.61415782]

[0. 1. 0. ... 0. 0.55319149 0.40483525]

...

[0. 0. 0. ... 0. 0.78723404 0.]

[0. 0. 0. ... 0. 0.46808511 0.40483525]

[0. 0. 0. ... 0. 0.85106383 0.40483525]]

EXP NO:15 EXPERIMENT TO UNDERSTAND THE DATA PREPROCESSING IN DATA SCIENCE

CODE :

```
import numpy as np

import pandas as pd

df = pd.read_csv("pre-process_datasample.csv")

df["Country"] = df["Country"].fillna(df["Country"].mode()[0])

df["Age"] = df["Age"].fillna(df["Age"].median())

df["Salary"] = df["Salary"].fillna(round(df["Salary"].mean()))

dummies = pd.get_dummies(df["Country"])

updated_dataset = pd.concat([dummies, df.iloc[:, [1, 2, 3]]], axis=1)

updated_dataset["Purchased"] =
updated_dataset["Purchased"].replace(["No", "Yes"], [0, 1])

print(updated_dataset)
```

OUTPUT :

Country Age Salary Purchased 0 France 44.0 72000.0 No 1 Spain 27.0 48000.0 2 Germany 30.0 54000.0 3 Spain 38.0 61000.0 Yes No No 4 Germany 40.0 NaN Yes 5 France 35.0 58000.0 Yes 6 Spain NaN 52000.0 7 France 48.0 79000.0 8 Germany 50.0 83000.0 9 France 37.0 67000.0 No Yes No Yes <class 'pandas.core.frame.DataFrame'> RangeIndex: 10 entries, 0 to 9 Data columns (total 4 columns): # Column Non-Null Count Dtype --- ------ -------------- ----- 0 Country 10 non-null object 1 Age 9 non-null float64 2 Salary 9 non-null float64 3 Purchased 10 non-null object dtypes: float64(2),

object(2) memory usage: 452.0+ bytes None 0 France Name: Country, dtype: object France <class 'pandas.core.series.Series'> Country Age Salary Purchased 0 France 44.0 72000.0 1 Spain 27.0 48000.0 No Yes 2 Germany 30.0 54000.0 3 Spain 38.0 61000.0 4 Germany 40.0 63778.0 No No Yes 5 France 35.0 58000.0 6 Spain 38.0 52000.0 7 France 48.0 79000.0 8 Germany 50.0 83000.0 9 France 37.0 67000.0 France Germany Spain 0 True False False 1 False False True 2 False True False 3 False False True 4 False True False 5 True False False 6 False False True 7 True False False 8 False True False 9 True False False Yes No Yes No Yes France Germany Spain Age Salary Purchased 0 True False False 44.0 72000.0 No 1 False False True 27.0 48000.0 Yes 2 False True False 30.0 54000.0 No 3 False False True 38.0 61000.0 No 4 False True False 40.0 63778.0 Yes 5 True False False 35.0 58000.0 Yes 6 False False True 38.0 52000.0 No 7 True False False 48.0 79000.0 Yes 8 False True False 50.0 83000.0 No 9 True False False 37.0 67000.0 Yes <class 'pandas.core.frame.DataFrame'> RangeIndex: 10 entries, 0 to 9 Data columns (total 4 columns): # Column Non-Null Count Dtype --- ------ -------------- ----- 0 Country 10 non-null object 1 Age 10 non-null float64 2 Salary 10 non-null float64 3 Purchased 10 non-null object dtypes: float64(2), object(2) memory usage: 452.0+ bytes None France Germany Spain Age Salary Purchased 0 True False False 44.0 72000.0 1 False False True 27.0 48000.0 2 False True False 30.0 54000.0 3 False False True 38.0 61000.0 0 1 0 0 4 False True False 40.0 63778.0 1 5 True False False 35.0 58000.0 1 6 False False True 38.0 52000.0 0 7 True False False 48.0 79000.0

1 8 False True False 50.0 83000.0 0 9 True False False 37.0 67000.0 1

EXP NO:16 EXPERIMENT TO UNDERSTAND EDA-QUANTITATIVE AND QUALITATIVE ANALYSIS

CODE :

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

tips = sns.load_dataset('tips')

print(tips.head())

sns.displot(tips.total_bill, kde=True)
sns.displot(tips.total_bill, kde=False)

sns.jointplot(x=tips.tip, y=tips.total_bill)
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")

sns.pairplot(tips)
sns.pairplot(tips, hue='time')
sns.pairplot(tips, hue='day')
```

```
sns.heatmap(tips.corr(numeric_only=True), annot=True)

sns.boxplot(x=tips.total_bill)
sns.boxplot(x=tips.tip)

sns.countplot(x=tips.day)
sns.countplot(x=tips.sex)

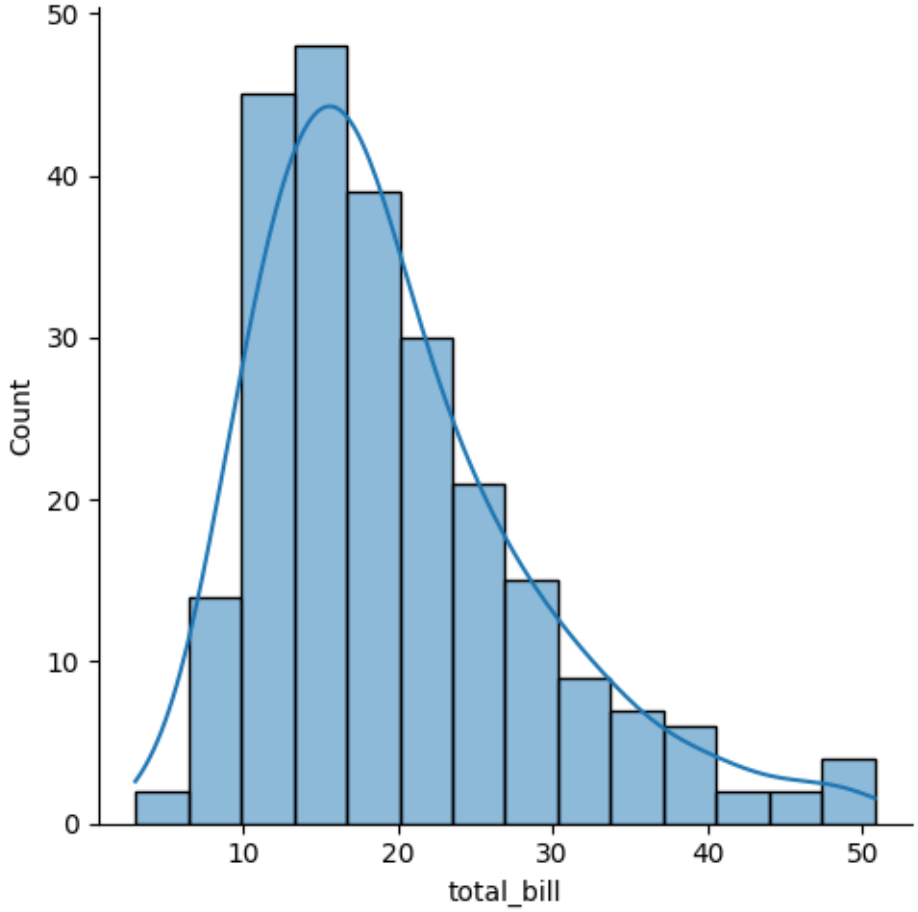
tips.sex.value_counts().plot(kind='pie')
tips.sex.value_counts().plot(kind='bar')

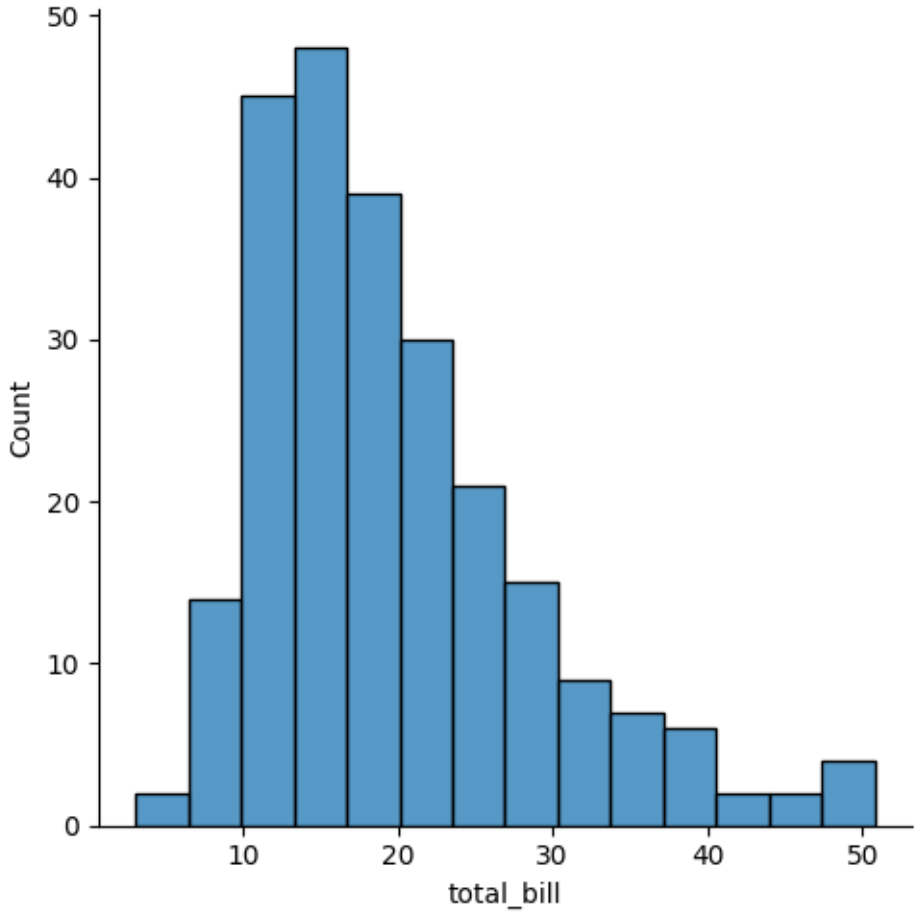
sns.countplot(x=tips[tips.time == 'Dinner']['day'])
plt.show()
```

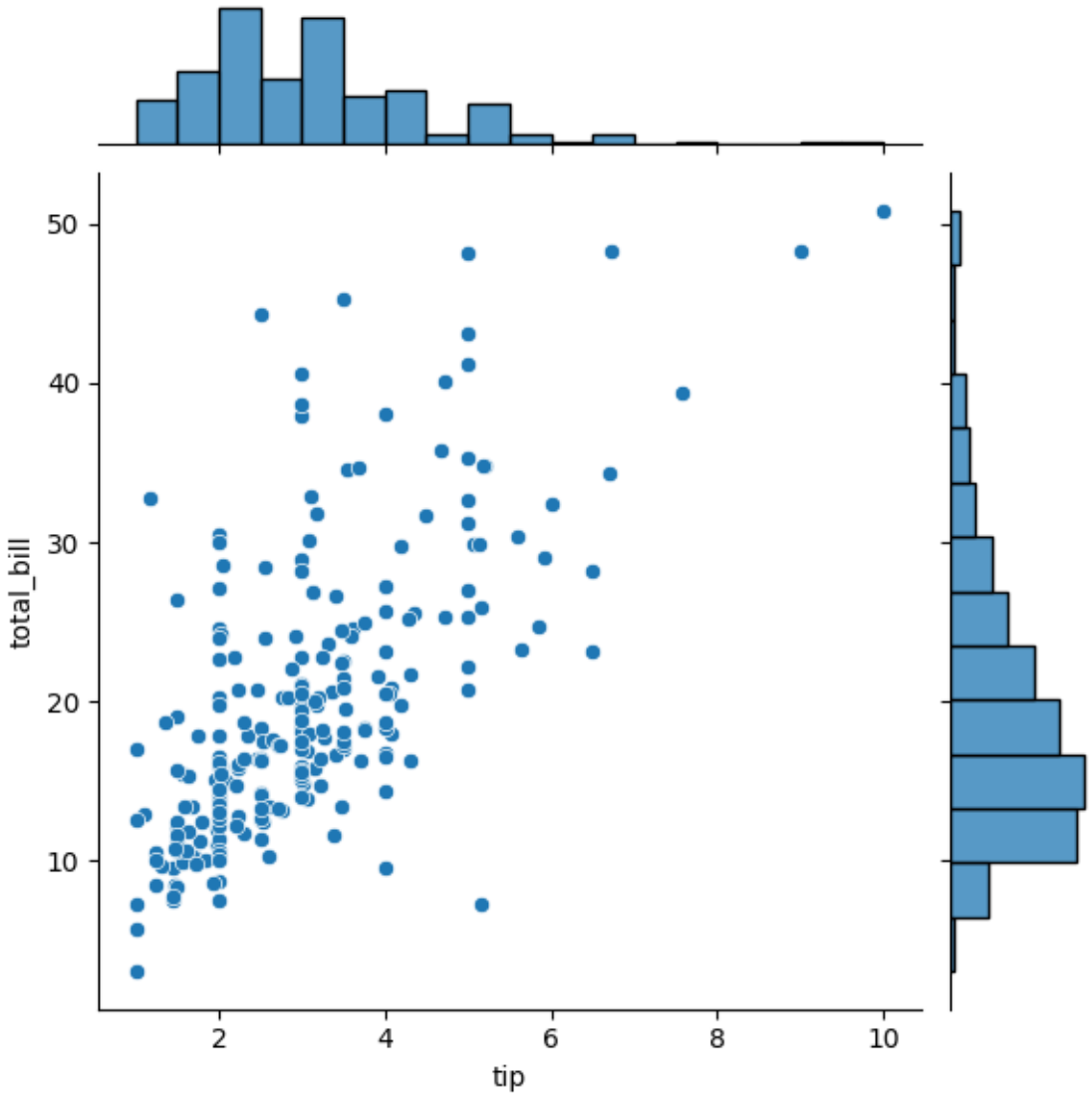
OUTPUT :

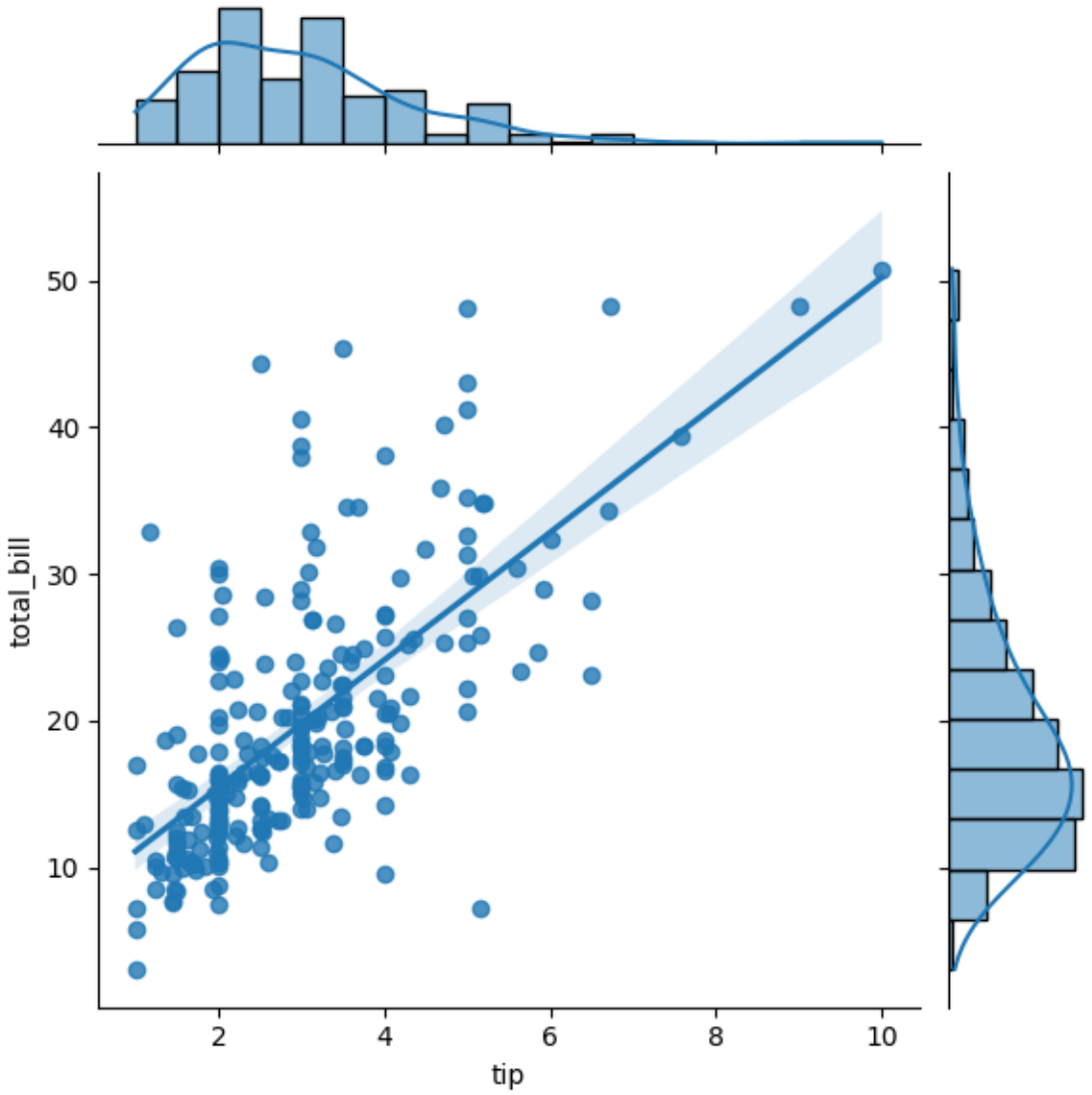
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

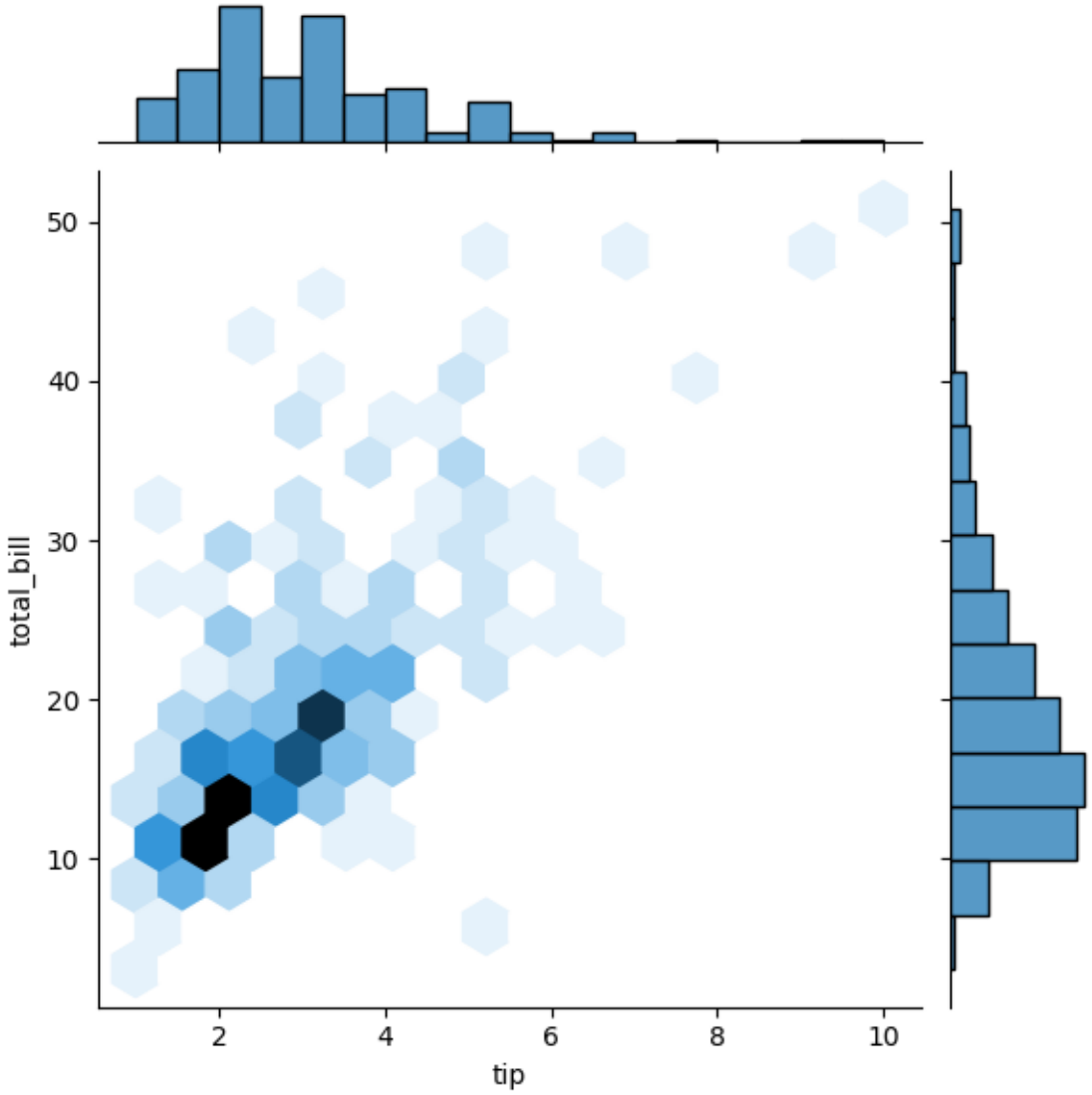
WARNING:matplotlib.axes._base:Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.

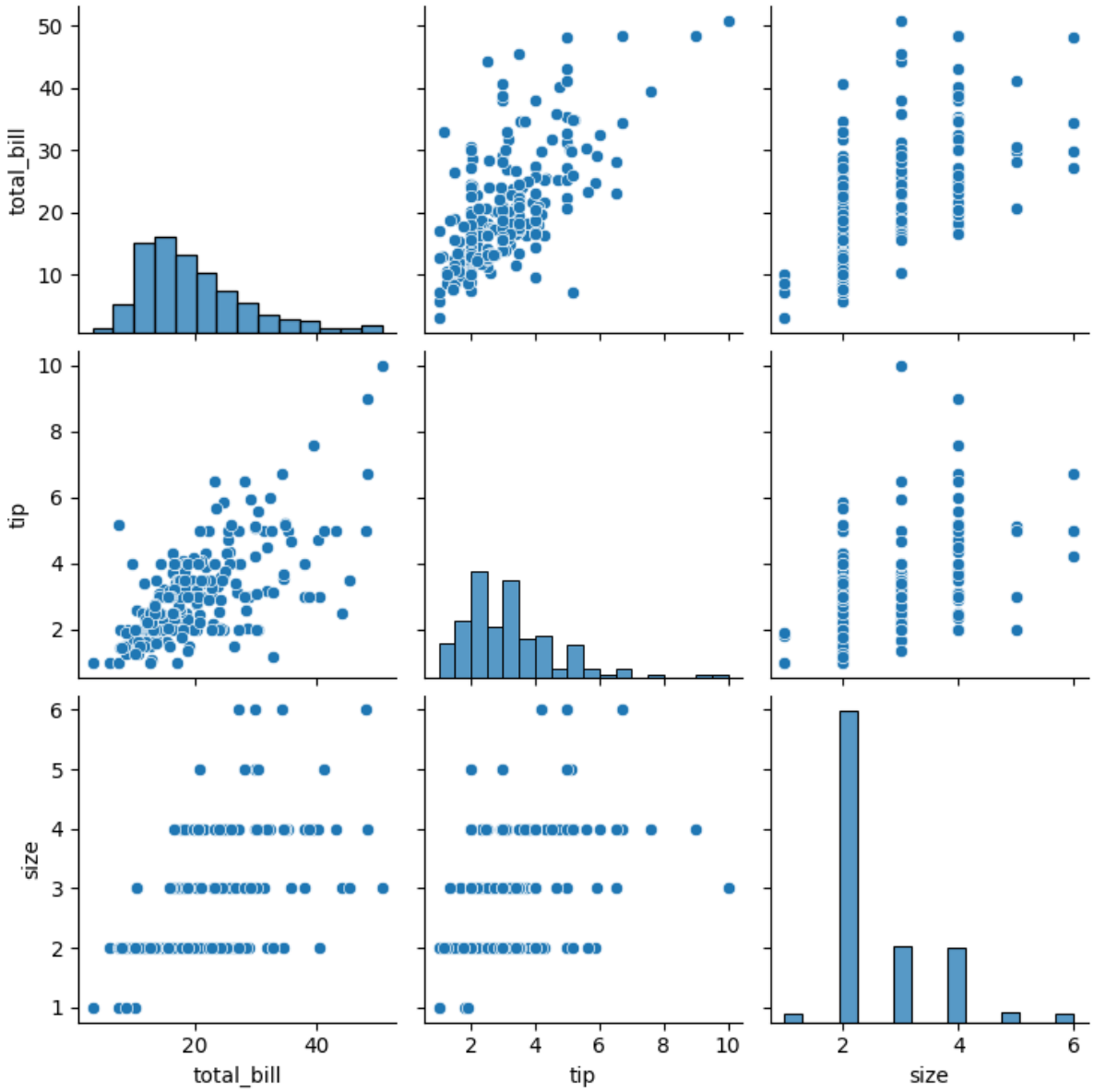


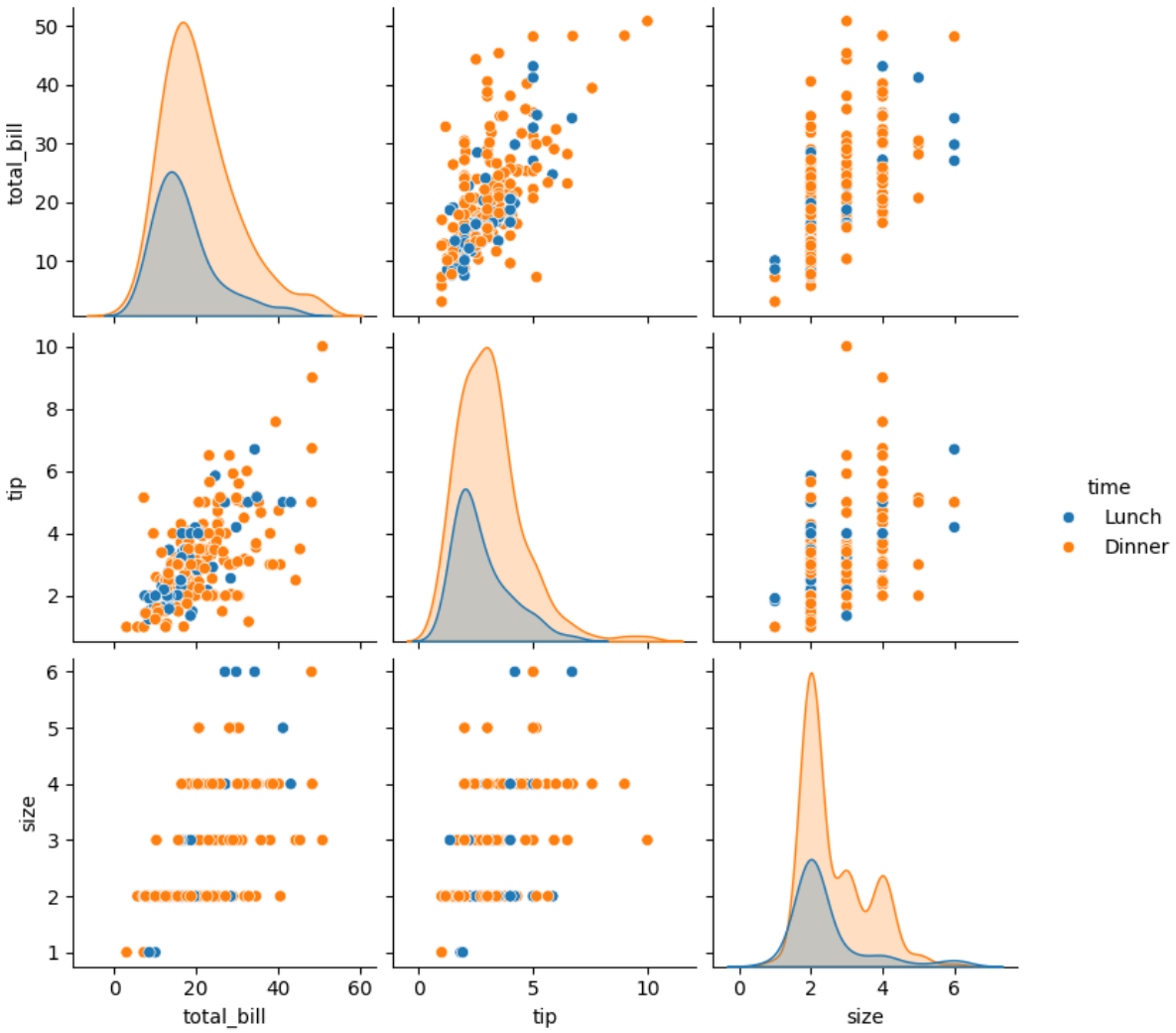

50
40
30
20
10
0
Count
10
20
30
40
50
total_bill


total_bill
tip
50
40
30
20
10
2
4
6
8
10

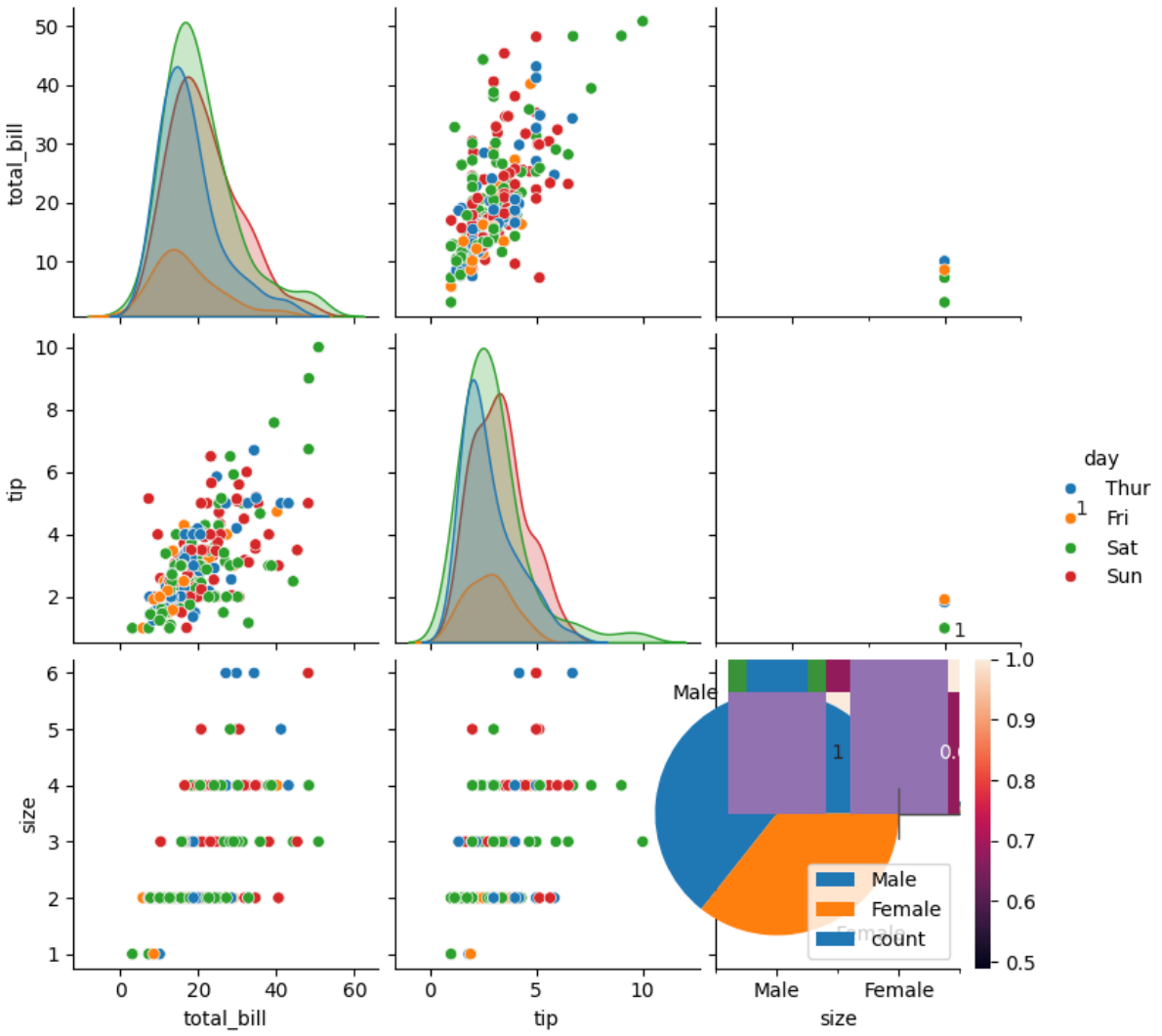

total_bill
50
40
30
20
10
2
4
6
8
10
tip


50
40
30
20
10
total_bill
2
4
6
8
10
tip


total_bill
tip
size
50
40
30
20
10
10
8
6
4
2
6
5
4
3
2
1
20
40
2
4
6
8
10
2
4
6
total_bill
tip
size



WARNING:matplotlib.axes._base:Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.


total_bill
tip
size
day
Thur
Fri
Sat
Sun
Male
Female
count

EXP NO:17 EXPERIMENT TO UNDERSTAND LINEAR REGRESSION FOR A GIVEN DATA SET

CODE :

```
import numpy as np
import pandas as pd
import pickle
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
df = pd.read_csv('Salary_data.csv')
print("Before removing missing values:")
df.info()
df.dropna(inplace=True)
print("\nAfter removing missing values:")
df.info()
print(df.describe())
features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values
x_train, x_test, y_train, y_test = train_test_split(
    features,
    label,
    test_size=0.2,
    random_state=42
)
```

```
model = LinearRegression()
model.fit(x_train, y_train)
print("\nTraining R² score:", model.score(x_train, y_train))
print("Testing R² score:", model.score(x_test, y_test))
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
with open('/content/SalaryPred.model', 'wb') as file:
    pickle.dump(model, file)
with open('/content/SalaryPred.model', 'rb') as file:
    model = pickle.load(file)
yr_of_exp = float(input("Enter Years of Experience: "))
yr_of_exp_np = np.array([[yr_of_exp]])
salary = model.predict(yr_of_exp_np)
print(
    f"Estimated salary for {yr_of_exp} years of experience "
    f"is ₹{salary[0][0]:,.2f}"
)
```

OUTPUT :

```
Before removing missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 2 columns):
```

```
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   YearsExperience  296 non-null    float64
 1   Salary          296 non-null    float64
dtypes: float64(2)
memory usage: 4.8 KB

After removing missing values:
<class 'pandas.core.frame.DataFrame'>
Index: 292 entries, 0 to 299
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   YearsExperience  292 non-null    float64
 1   Salary          292 non-null    float64
dtypes: float64(2)
memory usage: 6.8 KB
       YearsExperience         Salary
count       292.000000     292.000000
mean          7.839384   65975.287671
std           4.076998   22387.172902
min           0.300000   18000.000000
25%           4.575000   49128.500000
```

50% 7.900000 65040.000000

75% 11.200000 82683.500000

max 15.000000 121956.000000

Training R^2 score: 0.907767542415078

Testing R^2 score: 0.9098570405486877

Coefficient: [[5245.51746903]]

Intercept: [24614.15597493]

Enter Years of Experience: 3

Estimated salary for 3.0 years of experience is ₹40,350.71

EXP NO:18 EXPERIMENT TO UNDERSTAND KNN ALGORITHM FOR A GIVEN DATASET

CODE :

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
data=pd.read_csv('/content/Iris_Dataset.csv')
data.info()
df.variety.value_counts()
df.head()
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,ra
ndo
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
from sklearn.metrics import confusion_matrix
```

```
confusion_matrix(label,model_KNN.predict(features))

from sklearn.metrics import classification_report

print(classification_report(label,model_KNN.predict(features)))
```

OUTPUT :

<class 'pandas.core.frame.DataFrame'> RangeIndex: 150 entries, 0 to 149 Data columns (total 5 columns): # Column Non-Null Count Dtype --- ------ -------------- ----- 0 sepal.length 150 non-null float64 1 sepal.width 150 non-null float64 2 petal.length 150 non-null float64 3 petal.width 150 non-null float64 4 variety 150 non-null object dtypes: float64(4), object(1) memory usage: 6.0+ KB None variety Setosa 50 Versicolor 50 Virginica 50 Name: count, dtype: int64 sepal.length sepal.width petal.length petal.width variety 0 5.1 3.5 1.4 0.2 Setosa 1 4.9 3.0 1.4 0.2 Setosa 2 4.7 3.2 1.3 0.2 Setosa 3 4.6 3.1 1.5 0.2 Setosa 4 5.0 3.6 1.4 0.2 Setosa 0.9666666666666667 1.0 [[50 0 0] [0 47 3] [0 1 49]] precision recall f1-score support Setosa 1.00 1.00 1.00 50 Versicolor 0.98 0.94 0.96 50 Virginica 0.94 0.98 0.96 50 accuracy 0.97 150 macro avg 0.97 0.97 0.97 150 weighted avg 0.97 0.97 0.97 150

EXP NO:19EXPERIMENT TO UNDERSTAND LOGISTIC REGRESSION FOR A GIVEN DATA SET

CODE :

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
df = pd.read_csv("Social_Network_Ads.csv")
print(df.head())
print(df.info())
features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values
for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features,
        label,
        test_size=0.2,
        random_state=i,
        stratify=label
    )
```

```
    model = make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=1000)
    )
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    if test_score > train_score:
        print(
            "Test {:.4f}, Train {:.4f}, Random State {}".format(
                test_score, train_score, i
            )
        )
        break
random_state = i
x_train, x_test, y_train, y_test = train_test_split(
    features,
    label,
    test_size=0.2,
    random_state=random_state,
    stratify=label
)
finalModel = make_pipeline(
```

```
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)
finalModel.fit(x_train, y_train)
print("\nTraining Accuracy:", finalModel.score(x_train, y_train))
print("Testing Accuracy:", finalModel.score(x_test, y_test))
y_pred = finalModel.predict(x_test)
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

OUTPUT :

```
   User ID  Gender  Age  EstimatedSalary  Purchased
0    10001    Male   54            15437          1
1    10002    Male   25            64520          1
2    10003  Female   19            66748          0
3    10004  Female   45            71309          1
4    10005  Female   33            51826          0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
```

```
 0   User ID          800 non-null    int64
 1   Gender           800 non-null    object
 2   Age              800 non-null    int64
 3   EstimatedSalary  800 non-null    int64
 4   Purchased        800 non-null    int64
dtypes: int64(4), object(1)
memory usage: 31.4+ KB
None
```

Test 0.7688, Train 0.7594, Random State 2

Training Accuracy: 0.759375

Testing Accuracy: 0.76875

Confusion Matrix:

```
[[52 20]
 [17 71]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.72	0.74	72
1	0.78	0.81	0.79	88

accuracy			0.77	160
macro avg	0.77	0.76	0.77	160
weighted avg	0.77	0.77	0.77	160

EXP NO:20 EXPERIMENT TO UNDERSTAND K-MEANS CLUSTERING ALGORITHM FOR A GIVEN DATA SET

CODE :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
df=pd.read_csv('Mall_Customers.csv')
df.info()
df.head()
sns.pairplot(df)
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
```

```
.add_legend();
plt.show()
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
model=KMeans(n_clusters=i)
model.fit(features_el)
wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

OUTPUT :

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              600 non-null    int64
 1   Gender                  600 non-null    object
 2   Age                     600 non-null    int64
 3   Annual Income (k$)      600 non-null    int64
 4   Spending Score (1-100)  600 non-null    int64
dtypes: int64(4), object(1)
```

memory usage: 23.6+ KB

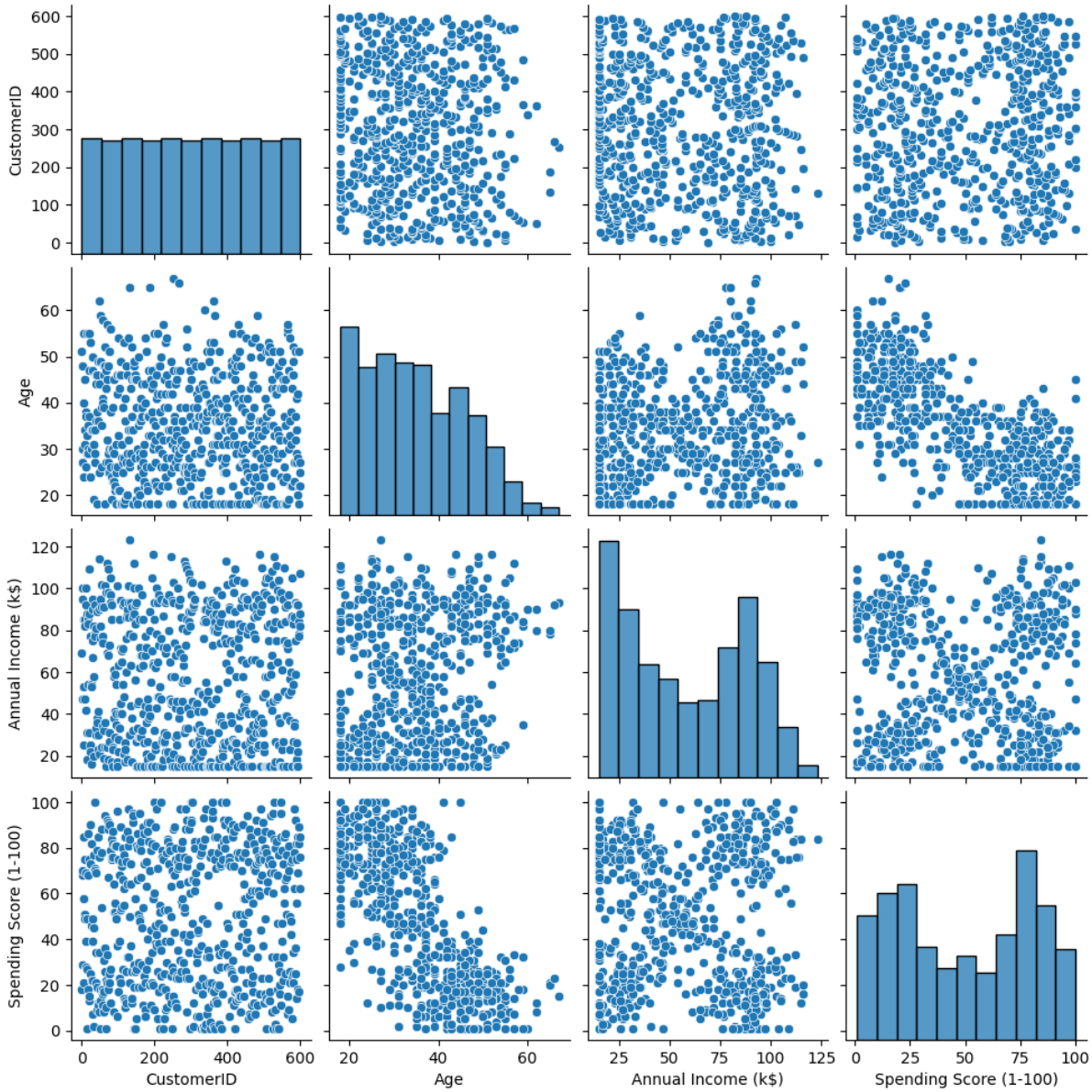
/tmp/ipykernel_6488/228583201.py:16: SettingWithCopyWarning:

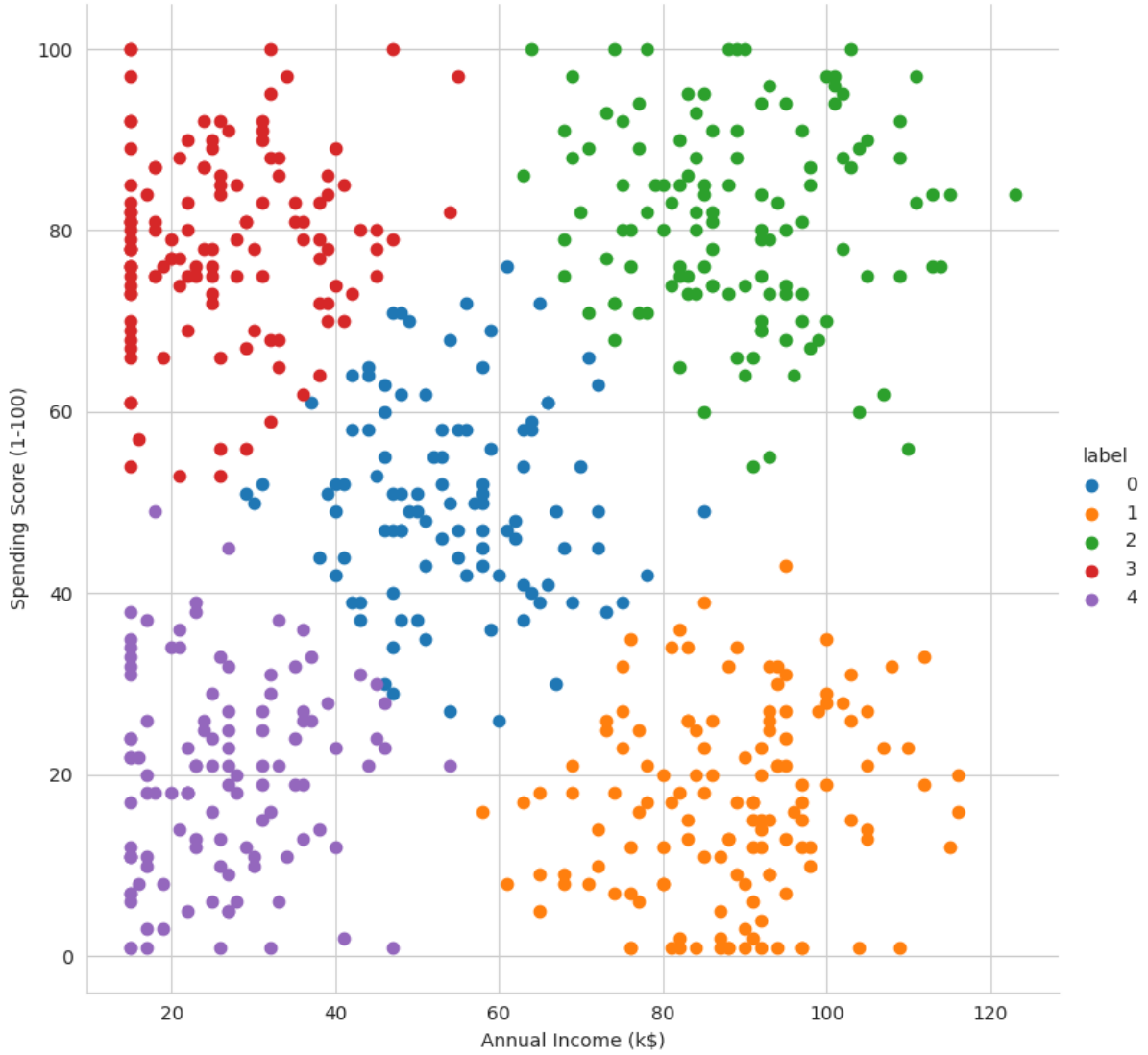
A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

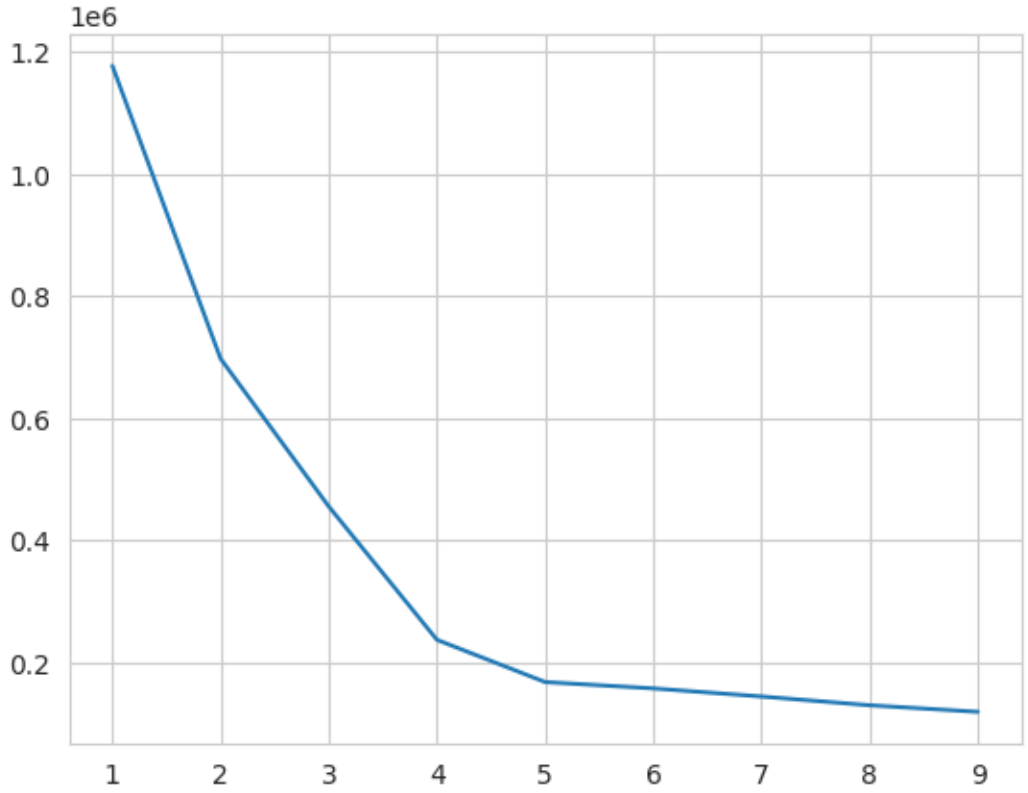
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
  Final['label']=model.predict(features)
```


CustomerID
Age
Annual Income (k$)
Spending Score (1-100)
0
200
400
600
20
40
60
25
50
75
100
125
0
25
50
75
100



[<matplotlib.lines.Line2D at 0x7e34dc269a90>]


1e6
1.2
1.0
0.8
0.6
0.4
0.2
1
2
3
4
5
6
7
8
9